

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования

«ТОМСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ СИСТЕМ
УПРАВЛЕНИЯ И РАДИОЭЛЕКТРОНИКИ» (ТУСУР)
Кафедра безопасности информационных систем (БИС)

МОДЕЛЬ ДЛЯ РАСПОЗНАВАНИЯ ДИЗАЙНА ЗНАЧКОВ
ОПЕРАЦИОННЫХ СИСТЕМ

Пояснительная записка курсовой работы по дисциплине
«Нейронные сети в обработке изображений и текста»

Студент гр. 744-1

_____ Т.А. Сулейманов

____.____.2026 г.

Руководитель

Преподаватель каф. БИС

_____ П.Ю. Лаптев

оценка _____.____.2026 г.

Министерство науки и высшего образования
РФ ФГБОУ ВО «Томский государственный университет систем управления и
радиоэлектроники»

ТОМСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ СИСТЕМ
УПРАВЛЕНИЯ И РАДИОЭЛЕКТРОНИКИ (ТУСУР)

Кафедра безопасности информационных систем (БИС)

УТВЕРЖДАЮ

И.О. Зав. Кафедрой БИС

_____ Е.Ю. Костюченко

____.____.2026 г.

Задание

на курсовую работу по дисциплине «Нейронные сети в обработке изображений и текста» студенту группы 744-1 факультета безопасности Т.А. Сулейманову.

1. Тема работы: «Модель для распознавания дизайна значков операционных систем».

2. Срок сдачи законченной работы «___» _____ 2026 г.

3. Исходные данные к работе:

3.1 Набор данных, содержащий суммарно не менее 700 стандартных значков операционных систем Windows, Mac OS X (macOS) и iPhone OS (iOS) различных версий, включая: Windows 3.0, 98, Vista, 7, 10, 11; OS X 10.0 Cheetah, 10.5 Leopard, 10.9 Mavericks, 10.10 Yosemite, macOS 10.15 Catalina, 11 Big Sur, 26 Tahoe; а также iPhone OS, iOS 6, 7, 11, 18 и 26.

3.2 Размеченный датасет с указанием корпорации (Apple или Microsoft), конкретной ОС и стиля дизайна (Skeuomorphism, Flat, Glassmorphism, Retro).

4. В рамках курсовой работы необходимо:

4.1 Ознакомиться с собранными данными, включающими иконки графических интерфейсов, и предметной областью, связанной с эволюцией дизайна операционных систем.

4.2 Изучить методы анализа изображений, включая предварительную обработку (нормализацию, аугментацию), а также методы классификации изображений с использованием свёрточных нейронных сетей.

4.3 Привести собранный набор данных к единому формату (PNG, 128×128 пикселей) с нормализацией цветовых каналов, сбалансировать классы с помощью аугментации.

4.4 Провести обучение модели нейронной сети на основе архитектуры ResNet18, включающей свёрточные слои и механизмы регуляризации (Dropout), а также провести тестирование модели с использованием валидационной выборки.

4.5 Проанализировать полученные результаты, включая точность классификации, матрицу ошибок (confusion matrix), и сделать выводы о качестве работы системы и возможных направлениях улучшения.

5. Содержание пояснительной записки (перечень подлежащих проработке вопросов):

5.1 Титульный лист.

5.2 Реферат.

5.3 Задание на курсовую работу.

5.4 Основная часть:

5.4.1 Исследование тематики (виды графического дизайна, обзор методов классификации изображений)

5.4.2 Обработка набора данных (сбор иконок, разметка, балансировка, аугментация)

5.4.3 Обучение нейронной сети (архитектура ResNet18)

5.4.4 Проверка качества обучения с использованием валидационной выборки

5.5 Заключение.

5.6 Список литературы.

5.7 Приложение.

Руководитель: преподаватель кафедры БИС

П.Ю. Лаптев

«28» февраля 2026 г.

Задание принял: студент группы 744-1

Т.А. Сулейманов

«28» февраля 2026 г.

Реферат

Пояснительная записка содержит X страниц, Y рисунков.

Ключевые слова: НЕЙРОННЫЕ СЕТИ, ИСКУССТВЕННЫЙ ИНТЕЛЛЕКТ, КЛАССИФИКАЦИЯ ИЗОБРАЖЕНИЙ, РАСПОЗНАВАНИЕ СТИЛЕЙ, СТАНДАРТНЫЕ ЗНАЧКИ ГРАФИЧЕСКИХ ИНТЕРФЕЙСОВ, OPERATING SYSTEMS, RESNET18, АУГМЕНТАЦИЯ ДАННЫХ, СВЁРТОЧНЫЕ НЕЙРОННЫЕ СЕТИ.

В данной работе рассматривается задача автоматического распознавания стилей дизайна и версий операционных систем по стандартным значкам графических интерфейсов. Полученные результаты могут применяться для анализа эволюции дизайна программного обеспечения, классификации интерфейсов, а также в образовательных и исследовательских целях.

Целью работы является разработка и обучение нейронной сети, способной автоматически определять направление дизайна (Skeuomorphism, Flat, Glassmorphism, Retro), конкретный стиль и версию операционной системы (Windows или macOS / iOS) на основе изображения стандартного значка.

Для решения поставленной задачи необходимо решить следующие задачи:

1. Произвести сбор, анализ и предварительную обработку набора стандартных значков (сбор оригинальных значков различных версий Windows, Mac OS X (macOS) и iPhone OS (iOS), разметка данных, конвертация в единый формат, нормализация изображений, балансировка классов с помощью аугментации);

2. Разработать и обучить модель нейронной сети на основе архитектуры ResNet18 для классификации направлений дизайна, стилей и версий операционных систем, а также провести тестирование обученной модели;

3. Произвести тестирование и валидацию модели на отдельной выборке;
4. Оформить отчет о проделанной работе.

В процессе выполнения работы был собран и размечен набор данных, включающий более 700 стандартных значков операционных систем, распределенных по 19 различным версиям, отражающим доминирующие направления дизайна. Стандартные значки были приведены к единому формату PNG (128×128 пикселей) с нормализацией цветовых каналов. Для балансировки классов и увеличения размера выборки применялась аугментация (повороты, отражения, изменение яркости, размытие).

Была разработана и обучена модель нейронной сети на основе свёрточной архитектуры ResNet18. Были получены три модели: для определения направления дизайна (4 класса), для определения стиля (7 классов) и для определения версии операционной системы (19 классов). Для анализа качества классификации были построены матрицы ошибок и графики обучения (потери, точность), а также проведен анализ уверенности предсказаний модели.

Содержание

Введение	8
1 Исследование тематики	9
2 Обработка набора данных	11
3 Подгрузка датасета	14
4 Работа модели	17
5 Загрузка пользовательских файлов для анализа	21
6 Пример работы модели	24
Заключение	38
Список использованных источников	39
ПРИЛОЖЕНИЕ А	40

Введение

Целью работы является разработка и обучение нейронной сети, способной автоматически определять направление дизайна (Skeuomorphism, Flat, Glassmorphism, Retro), конкретный стиль и версию операционной системы (Windows, macOS, iOS) на основе изображения стандартного значка графического интерфейса.

Для решения поставленной задачи необходимо решить следующие задачи:

1. Произвести сбор, анализ и предварительную обработку набора стандартных значков (сбор оригинальных значков различных версий Windows и macOS, разметка данных по корпорациям, версиям ОС и стилям, конвертация в единый формат, нормализация изображений, балансировка классов с помощью аугментации);
2. Разработать и обучить модель нейронной сети на основе архитектуры ResNet18 для классификации направлений дизайна, стилей и версий операционных систем, а также провести тестирование обученной модели;
3. Произвести тестирование и валидацию модели на отдельной выборке;
4. Оформить отчет о проделанной работе.

1 Исследование тематики

Распознавание стилей дизайна и версий операционных систем по стандартным значкам графических интерфейсов является важной задачей в области компьютерного зрения и машинного обучения, особенно в контексте анализа эволюции пользовательских интерфейсов и автоматической классификации программного обеспечения. Графические интерфейсы операционных систем (Windows, macOS, iOS) прошли долгий путь развития, и каждый этап характеризуется уникальными дизайнерскими решениями: от скевоморфизма с реалистичными текстурами до плоского дизайна (Flat), неоморфизма и глассморфизма[4]. Автоматизация распознавания таких стилей может значительно улучшить понимание тенденций в дизайне, а также найти применение в системах автоматической каталогизации, архивации интерфейсов и образовательных целях.

Современные подходы к распознаванию изображений включают использование глубоких нейронных сетей, которые эффективно решают задачи классификации. Основные этапы таких систем включают:

1. Предварительная обработка изображений. Для анализа иконок применяются методы нормализации (приведение к единому размеру, нормализация цветовых каналов) и аугментации (повороты, отражения, изменения яркости и контраста) для увеличения размера выборки и борьбы с переобучением.

2. Извлечение признаков. Современные свёрточные нейронные сети (CNN), такие как ResNet, VGG, EfficientNet, автоматически извлекают иерархические признаки из изображений: от простых (края, углы) до сложных (формы, текстуры, паттерны).

3. Классификация. На основе извлеченных признаков полносвязные слои сети выполняют классификацию изображений, определяя

принадлежность к одному из классов (направление дизайна, конкретный стиль или версия ОС).

4. Оценка качества. Для анализа работы модели используются метрики: точность (accuracy), полнота (recall), F1-мера, а также матрица ошибок (confusion matrix), позволяющая визуализировать, какие классы модель путает между собой.

В рамках данной работы исследуются методы обработки изображений, включая нормализацию, аугментацию и балансировку классов, а также архитектуры нейронных сетей на основе ResNet18, используя язык программирования Python[5]. Целью исследования является разработка системы, способной классифицировать стандартные значки операционных систем по направлениям дизайна, стилям и версиям с наиболее возможной высокой точностью.

2 Обработка набора данных

Исходный набор данных был собран вручную и представляет собой коллекцию стандартных значков графических интерфейсов операционных систем Windows и macOS. Общее количество значков в наборе данных составляет более 700, распределенных по 19 различным версиям операционных систем: Windows 3.0, 98, Vista, 7, 10, 11; macOS 10.0 (Cheetah), 10.5 (Leopard), 10.9 (Mavericks), 10.10 (Yosemite), 10.15 (Catalina), 11 (Big Sur), 26 (Tahoe); а также мобильные ОС: iPhone OS, iOS 6, 7, 11, 18, 26. Каждый значок связан с конкретной версией ОС и имеет определённый стиль дизайна (Skeuomorphism, Flat, Glassmorphism, Retro).

Изначально значки были извлечены из системных файлов операционных систем (файлы .dll, .mun в Windows и ресурсы .icns в macOS) и собраны из открытого архива иконок GUIs.org[2] – Graphical User Interface storehouse. Все значки являются стандартными – они входят в стартовый пакет системы из коробки. Для обеспечения разнообразия данных использовались значки различных типов: папки, приложения, системные утилиты, элементы управления. Все значки были отсортированы по папкам, соответствующим каждой версии ОС, и размечены вручную по трем признакам: корпорация (Apple/Microsoft), версия операционной системы и стиль дизайна. Разметка записана отдельно в формате JSON. Структуры набора данных и разметки продемонстрированы на рисунках 2.1 и 2.2.

 iOS 6	6/14/2026 11:20 AM	File folder	
 iOS 7	6/14/2026 11:20 AM	File folder	
 iOS 11	6/14/2026 11:20 AM	File folder	
 iOS 18	6/14/2026 11:20 AM	File folder	
 iOS 26	6/14/2026 11:20 AM	File folder	
 iPhone OS	6/14/2026 11:20 AM	File folder	
 Mac OS X 10.0 Cheetah	6/14/2026 11:20 AM	File folder	
 Mac OS X 10.5 Leopard	6/14/2026 11:20 AM	File folder	
 Mac OS X 10.9 Mavericks	6/14/2026 11:20 AM	File folder	
 Mac OS X 10.10 Yosemite	6/14/2026 11:20 AM	File folder	
 macOS 10.15 Catalina	6/14/2026 11:20 AM	File folder	
 macOS 11 Big Sur	6/14/2026 11:20 AM	File folder	
 macOS 26 Tahoe	6/14/2026 11:20 AM	File folder	
 Windows 3.0	6/14/2026 11:20 AM	File folder	
 Windows 7	6/14/2026 11:20 AM	File folder	
 Windows 10	6/14/2026 11:20 AM	File folder	
 Windows 11	6/14/2026 11:20 AM	File folder	
 Windows 98	6/14/2026 11:20 AM	File folder	
 Windows Vista	6/14/2026 11:20 AM	File folder	
 labels_remapped.json	6/14/2026 1:59 PM	JSON File	137 KB

Рисунок 2.1 – структура папок датасета

```

4775 },
4776 {
4777     "file": "Windows 98\\Date and Time.png",
4778     "corporation": "Microsoft",
4779     "os": "Windows 98",
4780     "direction": "Retro",
4781     "style": "Windows 80-90s Retro"
4782 },
4783 {
4784     "file": "Windows 98\\Desktop.png",
4785     "corporation": "Microsoft",
4786     "os": "Windows 98",
4787     "direction": "Retro",
4788     "style": "Windows 80-90s Retro"
4789 },
4790 {
4791     "file": "Windows 98\\Display.png",
4792     "corporation": "Microsoft",
4793     "os": "Windows 98",
4794     "direction": "Retro",
4795     "style": "Windows 80-90s Retro"
4796 },
4797 {
4798     "file": "Windows 98\\Documents.png",
4799     "corporation": "Microsoft",
4800     "os": "Windows 98",
4801     "direction": "Retro",
4802     "style": "Windows 80-90s Retro"
4803 },
4804 {
4805     "file": "Windows 98\\Email.png",
4806     "corporation": "Microsoft",
4807     "os": "Windows 98",
4808     "direction": "Retro",
4809     "style": "Windows 80-90s Retro"
4810 },
4811 {
4812     "file": "Windows 98\\Empty Recycle Bin.png",
4813     "corporation": "Microsoft",
4814     "os": "Windows 98",
4815     "direction": "Retro",
4816     "style": "Windows 80-90s Retro"
4817 },
4818 {
4819     "file": "Windows 98\\Font TTF.png",
4820     "corporation": "Microsoft",
4821     "os": "Windows 98",

```

Рисунок 2.2 – структура JSON-разметки

3 Подгрузка датасета

Модель состоит из трёх последовательных классификаторов на основе архитектуры ResNet18: классификатор направления дизайна (4 класса), классификатор конкретного стиля (7 классов) и классификатор конкретной операционной системы (19 классов). После загрузки размеченного набора данных алгоритм определяет класс (направление дизайна, стиль или ОС в зависимости от загрузчика), содержащий наибольшее количество экземпляров. Алгоритм записывает это число, умножает его на два и затем проводит аугментацию всех классов, пока количество экземпляров в каждом классе не достигнет вычисленного целевого значения. Данный подход позволяет сбалансировать классы, применяя при этом как геометрическую, так и цветовую аугментацию. Аналогичный алгоритм используется для всех трёх признаков.

Реализация описанного алгоритма представлена в виде трёх функций (loadDirection, loadStyle и loadOS), каждая из которых отвечает за создание сбалансированных тренировочной и валидационной выборок для соответствующего классификатора. Общий вид функций загрузки данных приведён на рисунке 3.1.

```
import os
import json
import glob
import shutil
from pathlib import Path
from PIL import Image
import numpy as np
import matplotlib.pyplot as plt
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
from sklearn.model_selection import train_test_split
import torch
import torch.nn as nn
import torch.nn.functional as F
import torch.optim as optim
from torch.utils.data import Dataset, DataLoader, random_split
from torchvision import transforms, models
from torchvision.utils import make_grid
import random
import time
from datetime import datetime

def user():...

def loadDirection(labels_file, root_dir, batch_size=32, train_ratio=0.8, image_size=128):...

def loadStyle(labels_file, root_dir, batch_size=32, train_ratio=0.8, image_size=128):...

def loadOS(labels_file, root_dir, batch_size=32, train_ratio=0.8, image_size=128):...
```

Рисунок 3.1 – функции для подгрузки датасета, аугментации и создания выборок

Фрагмент реализации функции loadDirection(), отвечающей за загрузку данных для классификатора направлений дизайна, представлен на рисунке 3.2.

```
def loadDirection(labels_file, root_dir, batch_size=32, train_ratio=0.8, image_size=128):
    with open(labels_file, 'r', encoding='utf-8') as f:
        labels = json.load(f)

    direction_to_idx = {direction: i for i, direction in enumerate(sorted(set(item["direction"] for item in labels)))}
    idx_to_direction = {i: direction for direction, i in direction_to_idx.items()}
    num_directions = len(direction_to_idx)

    print(f"Загружено иконок: {len(labels)}")
    print(f"Направления дизайна: {list(direction_to_idx.keys())}")

    direction_groups = {}
    for item in labels:
        direction = item["direction"]
        if direction not in direction_groups:
            direction_groups[direction] = []
        direction_groups[direction].append(item)

    print(f"Разделение по направлениям:")
    for direction, items in direction_groups.items():
        print(f"  {direction}: {len(items)} иконок")

    max_count = max(len(items) for items in direction_groups.values())
    target_count = max_count * 2
    print(f"Максимальное количество иконок в одном направлении: {max_count}")
    print(f"Целевое количество для каждого направления (x2): {target_count}")

    def get_heavy_augmentation():
        return transforms.Compose([
            transforms.Resize((image_size, image_size)),
            transforms.RandomHorizontalFlip(p=0.5),
            transforms.RandomRotation(degrees=25),
            transforms.ColorJitter(brightness=0.4, contrast=0.4, saturation=0.4, hue=0.15),
            transforms.RandomAffine(degrees=15, translate=(0.2, 0.2), scale=(0.8, 1.2)),
            transforms.GaussianBlur(kernel_size=(3, 3), sigma=(0.1, 1.0)),
            transforms.ToTensor(),
            transforms.Normalize(mean=[0.485, 0.456, 0.406],
                                std=[0.229, 0.224, 0.225])
        ])

    balanced_items = []
    for style, items in style_groups.items():
        current_count = len(items)
        if current_count < target_count:
            copies_needed = target_count - current_count
            copies_per_item = copies_needed // current_count + 1

            print(f"style: {current_count} -> {target_count} (+{copies_needed} аугментаций)")

            for item in items:
                balanced_items.append((item, False))
                for _ in range(copies_per_item):
                    balanced_items.append((item, True))
            else:
                print(f"style: {current_count} -> {target_count} (без изменений)")
                for item in items:
                    balanced_items.append((item, False))

    random.shuffle(balanced_items)
```

Рисунок 3.2 – фрагмент функции loadDirection()

Фрагмент реализации функции loadStyle(), отвечающей за загрузку данных для классификатора стилей, представлен на рисунке 3.3.

```
def get_light_augmentation():
    return transforms.Compose([
        transforms.Resize((image_size, image_size)),
        transforms.RandomHorizontalFlip(p=0.3),
        transforms.RandomRotation(degrees=10),
        transforms.ColorJitter(brightness=0.15, contrast=0.15, saturation=0.15),
        transforms.ToTensor(),
        transforms.Normalize(mean=[0.485, 0.456, 0.406],
                              std=[0.229, 0.224, 0.225])
    ])

val_transform = transforms.Compose([
    transforms.Resize((image_size, image_size)),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406],
                        std=[0.229, 0.224, 0.225])
])

balanced_items = []

for style, items in style_groups.items():
    current_count = len(items)
    if current_count < target_count:
        copies_needed = target_count - current_count
        copies_per_item = copies_needed // current_count + 1

        print(f"style: {current_count} -> {target_count} (+{copies_needed} аугментаций)")

        for item in items:
            balanced_items.append((item, False))
            for _ in range(copies_per_item):
                balanced_items.append((item, True))
        else:
            print(f"style: {current_count} -> {target_count} (без изменений)")
            for item in items:
                balanced_items.append((item, False))

random.shuffle(balanced_items)
```

Рисунок 3.3 – фрагмент функции loadStyle()

Фрагмент реализации функции loadOS(), отвечающей за загрузку данных для классификатора операционных систем, представлен на рисунке 3.4.

```
def get_item(item_data, is_val=False):
    item, is_augmented = item_data
    img_path = os.path.join(root_dir, item["file"])

    try:
        image = Image.open(img_path).convert("RGB")
    except Exception:
        image = Image.new("RGB", (image_size, image_size), (0, 0, 0))

    if is_val:
        transform = val_transform
    elif is_augmented:
        transform = get_heavy_augmentation()
    else:
        transform = get_light_augmentation()

    image = transform(image)
    os_label = os_to_idx[item["os"]]

    return image, os_label

train_size = int(train_ratio * len(balanced_items))
val_size = len(balanced_items) - train_size

train_items = balanced_items[:train_size]
val_items = balanced_items[train_size:]

train_data = [get_item(item, is_val=False) for item in train_items]
val_data = [get_item(item, is_val=True) for item in val_items]

train_loader = DataLoader(train_data, batch_size=batch_size, shuffle=True)
val_loader = DataLoader(val_data, batch_size=batch_size, shuffle=False)

print(f"\nTrain (bq): {len(train_data)} иконок")
print(f"Val (bq): {len(val_data)} иконок")
```

Рисунок 3.4 – фрагмент функции loadOS()

4 Работа модели

Вся система представляет собой каскад из трёх нейросетевых классификаторов, каждый из которых построен на архитектуре ResNet18[6] и работает последовательно.

Первый классификатор определяет направление дизайна иконки. Он анализирует изображение и относит его к одной из четырёх визуальных эпох: скевоморфизм, плоский дизайн, гласморфизм или ретро-стиль. Этот классификатор обучен на 738 иконках каждого направления после балансировки и достигает точности около 82–88% на валидационной выборке. На этом уровне модель учится различать самые общие визуальные признаки: наличие объёмных теней и текстур (скевоморфизм), плоские минималистичные формы (Flat), эффект стекла и прозрачности (Glassmorphism) или пиксельную графику высокого разрешения (Retro). Фрагмент кода первого классификатора представлен на рисунке 4.1.

```

def findDirection():
    base_path = os.path.dirname(__file__)
    DATASET_PATH = os.path.join(base_path, "НСВОИИТ курсовая dataset_converted")
    LABELS_FILE = os.path.join(DATASET_PATH, "labels_remapped.json")
    MODEL_PATH = os.path.join(base_path, "direction_model_best.pth")

    BATCH_SIZE = 128
    EPOCHS = 30
    LEARNING_RATE = 0.001
    IMAGE_SIZE = 128
    DEVICE = torch.device("cuda" if torch.cuda.is_available() else "cpu")

    print(f"\nУстройство: {DEVICE}")

    if os.path.exists(MODEL_PATH):
        print(f"\nНайдена сохранённая модель: {MODEL_PATH}")
        answer = input("Загрузить сохранённую модель? (y/n): ")
        if answer.lower() == 'y':
            train_loader, val_loader, num_directions, idx_to_direction = loadDirection(
                labels_file=LABELS_FILE,
                root_dir=DATASET_PATH,
                batch_size=BATCH_SIZE,
                train_ratio=0.8,
                image_size=IMAGE_SIZE
            )

            direction_names = list(idx_to_direction.values())

            model = models.resnet18(weights=None)
            model.conv1 = nn.Conv2d(3, 64, kernel_size=7, stride=2, padding=3, bias=False)
            in_features = model.fc.in_features
            model.fc = nn.Linear(in_features, num_directions)
            model.load_state_dict(torch.load(MODEL_PATH, map_location=DEVICE))
            model = model.to(DEVICE)

            print(f"Модель загружена: {MODEL_PATH}")

    def predict(image):
        model.eval()
        transform = transforms.Compose([
            transforms.Resize((IMAGE_SIZE, IMAGE_SIZE)),
            transforms.ToTensor(),
            transforms.Normalize(mean=[0.485, 0.456, 0.406],
                                std=[0.229, 0.224, 0.225])
        ])

```

Рисунок 4.1 – первый классификатор

Второй классификатор определяет конкретный стиль иконки. Он работает с учётом результата первого классификатора: модель сначала получает предсказанное направление, а затем рассматривает только те стили, которые могут принадлежать этому направлению. Например, если первая модель определила направление «Flat», то вторая модель будет выбирать только из стилей «Apple Flat by Jony Ive», «Windows Fluent» и «Windows Metro», отсекая заведомо неподходящие варианты. Это позволяет значительно повысить точность классификации, так как модель не путается между стилями из разных направлений. Всего система распознаёт 7 стилей, каждый из которых привязан к конкретной корпорации (Apple или Microsoft). Фрагмент кода второго классификатора представлен на рисунке 4.2.

```

def findStyle():
    base_path = os.path.dirname(__file__)
    DATASET_PATH = os.path.join(base_path, "НСВОИИТ курсовая dataset_converted")
    LABELS_FILE = os.path.join(DATASET_PATH, "labels_remapped.json")
    MODEL_PATH = os.path.join(base_path, "style_model_best.pth")

    BATCH_SIZE = 128
    EPOCHS = 30
    LEARNING_RATE = 0.001
    IMAGE_SIZE = 128
    DEVICE = torch.device("cuda" if torch.cuda.is_available() else "cpu")

    # Словарь: направление → список стилей
    DIRECTION_TO_STYLES = {
        "Skeuomorphism": ["Apple Skeuomorphism", "Windows Aero"],
        "Flat": ["Apple Flat by Jony Ive", "Windows Fluent", "Windows Metro"],
        "Glassmorphism": ["Apple Liquid Glass"],
        "Retro": ["Windows 80-90s Retro"]
    }

    # Обратный словарь: стиль → направление
    STYLE_TO_DIRECTION = {}
    for direction, styles in DIRECTION_TO_STYLES.items():
        for style in styles:
            STYLE_TO_DIRECTION[style] = direction

    print(f"\nУстройство: {DEVICE}")

    if os.path.exists(MODEL_PATH):
        print(f"\nНайдена сохранённая модель: {MODEL_PATH}")
        answer = input("Загрузить сохранённую модель? (y/n): ")
        if answer.lower() == 'y':
            train_loader, val_loader, num_styles, idx_to_style = loadStyle(
                labels_file=LABELS_FILE,
                root_dir=DATASET_PATH,
                batch_size=BATCH_SIZE,
                train_ratio=0.8,
                image_size=IMAGE_SIZE
            )

            model = models.resnet18(weights=None)
            model.conv1 = nn.Conv2d(3, 64, kernel_size=7, stride=2, padding=3, bias=False)
            in_features = model.fc.in_features
            model.fc = nn.Linear(in_features, num_styles)
            model.load_state_dict(torch.load(MODEL_PATH, map_location=DEVICE))
            model = model.to(DEVICE)

```

Рисунок 4.2 – второй классификатор

Третий классификатор определяет конкретную версию операционной системы. Он также использует результат предыдущего уровня: после того как определён стиль, модель анализирует иконку и выбирает наиболее вероятную версию ОС из числа тех, которые соответствуют этому стилю. Например, если определён стиль «Apple Skeuomorphism», то классификатор будет выбирать из iPhone OS, iOS 6, Mac OS X 10.0 Cheetah, 10.5 Leopard и 10.9 Mavericks. Всего система распознаёт 19 различных версий операционных систем Windows, macOS и iOS. Фрагмент кода третьего классификатора представлен на рисунке 4.3.

```

def findOS():
    base_path = os.path.dirname(__file__)
    DATASET_PATH = os.path.join(base_path, "НСВОИИТ курсовая датасет_converted")
    LABELS_FILE = os.path.join(DATASET_PATH, "labels_remapped.json")
    MODEL_PATH = os.path.join(base_path, "os_model_best.pth")

    BATCH_SIZE = 128
    EPOCHS = 30
    LEARNING_RATE = 0.001
    IMAGE_SIZE = 128
    DEVICE = torch.device("cuda" if torch.cuda.is_available() else "cpu")

    # Словарь: стиль → список ОС
    STYLE_TO_OS = {
        "Apple Skeuomorphism": [
            "iPhone OS",
            "iOS 6",
            "Mac OS X 10.0 Cheetah",
            "Mac OS X 10.5 Leopard",
            "Mac OS X 10.9 Mavericks"
        ],
        "Windows Aero": [
            "Windows Vista",
            "Windows 7"
        ],
        "Apple Flat by Jony Ive": [
            "iOS 7",
            "iOS 11",
            "iOS 18",
            "Mac OS X 10.10 Yosemite",
            "macOS 10.15 Catalina",
            "macOS 11 Big Sur"
        ],
        "Windows Fluent": [
            "Windows 11"
        ],
        "Windows Metro": [
            "Windows 10"
        ],
        "Apple Liquid Glass": [
            "iOS 26",
            "macOS 26 Tahoe"
        ],
        "Windows 80-90s Retro": [
            "Windows 3.0",

```

Рисунок 4.3 – третий классификатор

Таким образом, три классификатора работают в иерархической связке: первая модель даёт грубую оценку (направление), вторая уточняет (стиль), третья детализирует до конкретной версии ОС. Такая каскадная архитектура позволяет эффективно решать задачу многоклассовой классификации, разбивая её на несколько более простых подзадач.

5 Загрузка пользовательских файлов для анализа

Чтобы модель проанализировала пользовательские файлы на направление дизайна, конкретный стиль и принадлежность к конкретной ОС, была написана функция `user()`. Функция `user()` предназначена для загрузки и предварительной обработки пользовательских иконок, которые необходимо классифицировать. Она работает полностью автономно и не требует вмешательства пользователя.

При первом вызове функция создаёт папку `User` в той же директории, где находится исполняемый скрипт. Если папка уже существует, она просто использует её. Пользователь должен поместить в эту папку иконки в любом из поддерживаемых форматов: PNG, JPG, JPEG, ICO, SVG, ICNS, GIF, BMP, TIFF или WEBP.

После того как иконки размещены, функция сканирует содержимое папки `User`, игнорируя вложенные директории, и отбирает все файлы с поддерживаемыми расширениями. Для каждого найденного изображения выполняется конвейер предварительной обработки.

Сначала изображение приводится к цветовой модели RGB. Если исходная иконка имеет прозрачность (форматы PNG с альфа-каналом, ICO и другие), прозрачные области заменяются на белый фон, так как нейросеть ожидает на вход трёхцветное изображение без альфа-канала. Затем иконка изменяется до размера 128×128 пикселей. При этом важно сохранить пропорции исходного изображения: функция вычисляет коэффициент масштабирования так, чтобы иконка вписалась в квадрат 128×128, после чего добавляет белые поля по краям для заполнения пустого пространства. Такой подход позволяет избежать искажения формы значка.

Обработанные изображения сохраняются в подпапку `converted` внутри папки `User` с добавлением суффикса `_converted` в имени файла и расширением PNG. Сама функция возвращает два списка: список обработанных

изображений в виде объектов PIL Image, готовых к передаче в классификаторы, и список путей к сохранённым файлам для информационных целей.

Если в папке User не оказывается ни одного поддерживаемого изображения, функция выводит соответствующее сообщение и возвращает пустые списки, позволяя основной программе корректно обработать эту ситуацию. Фрагмент кода функции user() и папка User представлены на рисунках 5.1 и 5.2.

```
def user():
    base_path = os.path.dirname(__file__)
    user_folder = os.path.join(base_path, "User")

    if not os.path.exists(user_folder):
        os.makedirs(user_folder)
        print(f"Создана папка: {user_folder}")
        print("Положите туда иконки и запустите снова")
        return [], []

    supported_extensions = {'.png', '.jpg', '.jpeg', '.ico', '.svg', '.icns', '.gif', '.bmp', '.tiff', '.webp'}

    converted_folder = os.path.join(user_folder, "converted")
    os.makedirs(converted_folder, exist_ok=True)

    image_files = []

    for filename in os.listdir(user_folder):
        file_path = os.path.join(user_folder, filename)

        if os.path.isdir(file_path):
            continue

        ext = os.path.splitext(filename)[1].lower()
        if ext not in supported_extensions:
            continue

        image_files.append(file_path)

    if not image_files:
        print("В папке User не найдено изображений")
        print(f"Поддерживаются: {supported_extensions}")
        return [], []

    print(f"Найдено изображений: {len(image_files)}")

    images = []
    image_paths = []

    for img_path in image_files:
        try:
            with Image.open(img_path) as img:
                if img.mode in ('RGBA', 'LA', 'P'):
                    background = Image.new("RGB", img.size, (255, 255, 255))
                    if img.mode == 'P':
```

Рисунок 5.1 – функция user()

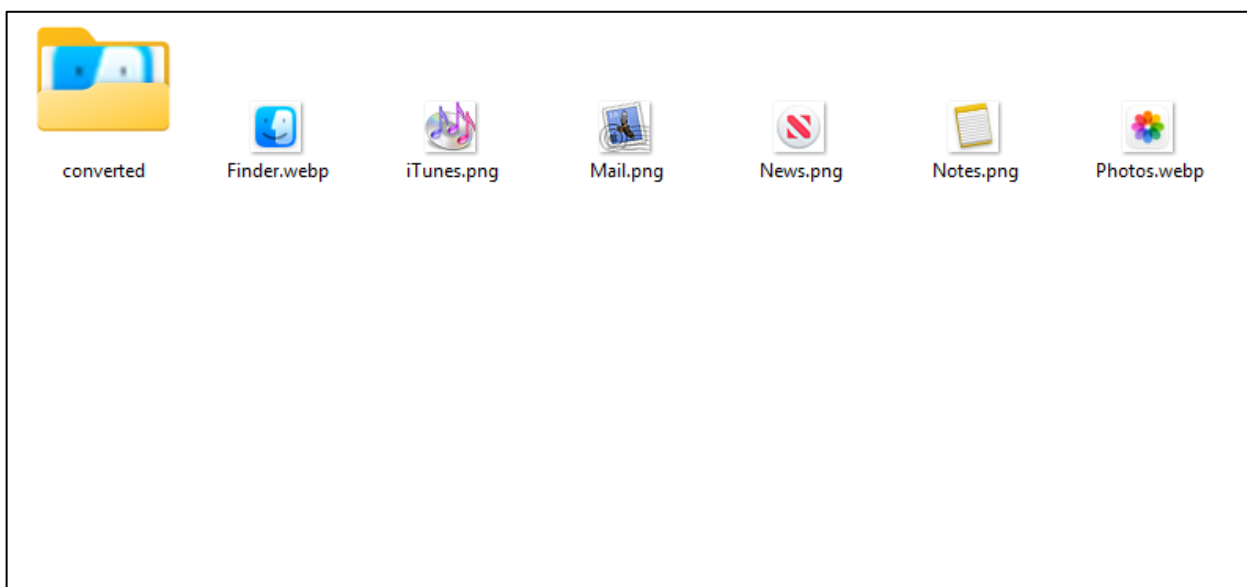


Рисунок 5.2 – папка User с тестовыми значками

6 Пример работы модели

Для начала работы в папку User были загружены 6 тестовых значков: Finder из macOS 26 Tahoe, iTunes из OS X 10.0 Cheetah, Mail из OS X 10.5 Leopard, News из macOS 10.15 Catalina, Notes из OS X 10.10 Yosemite и Photos из iOS 26 (рисунок 6.1).

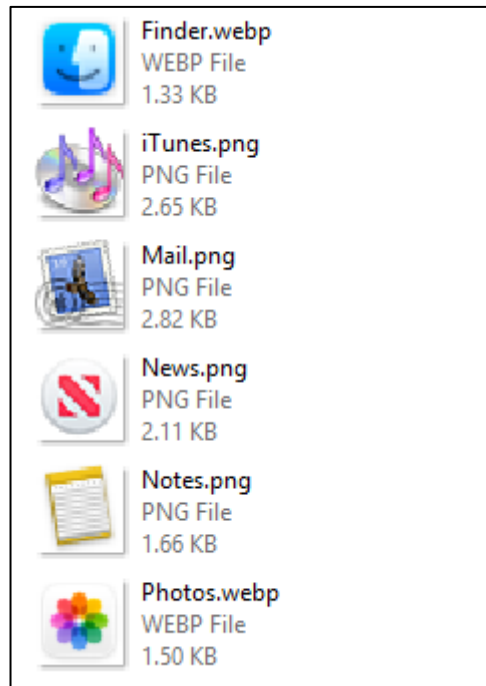


Рисунок 6.1 – пользовательские значки

Подрузка пользовательских значков произойдёт после обучения модели. Классификаторы обучаются один за другим. В процессе обучения выявляется лучший классификатор, который затем применяется для анализа пользовательских значков. Для обучения классификатора направления дизайна использовалась выборка, сбалансированная до 738 иконок на каждый из четырёх классов. Распределение экземпляров классов в валидационной выборке представлено на рисунке 6.2.

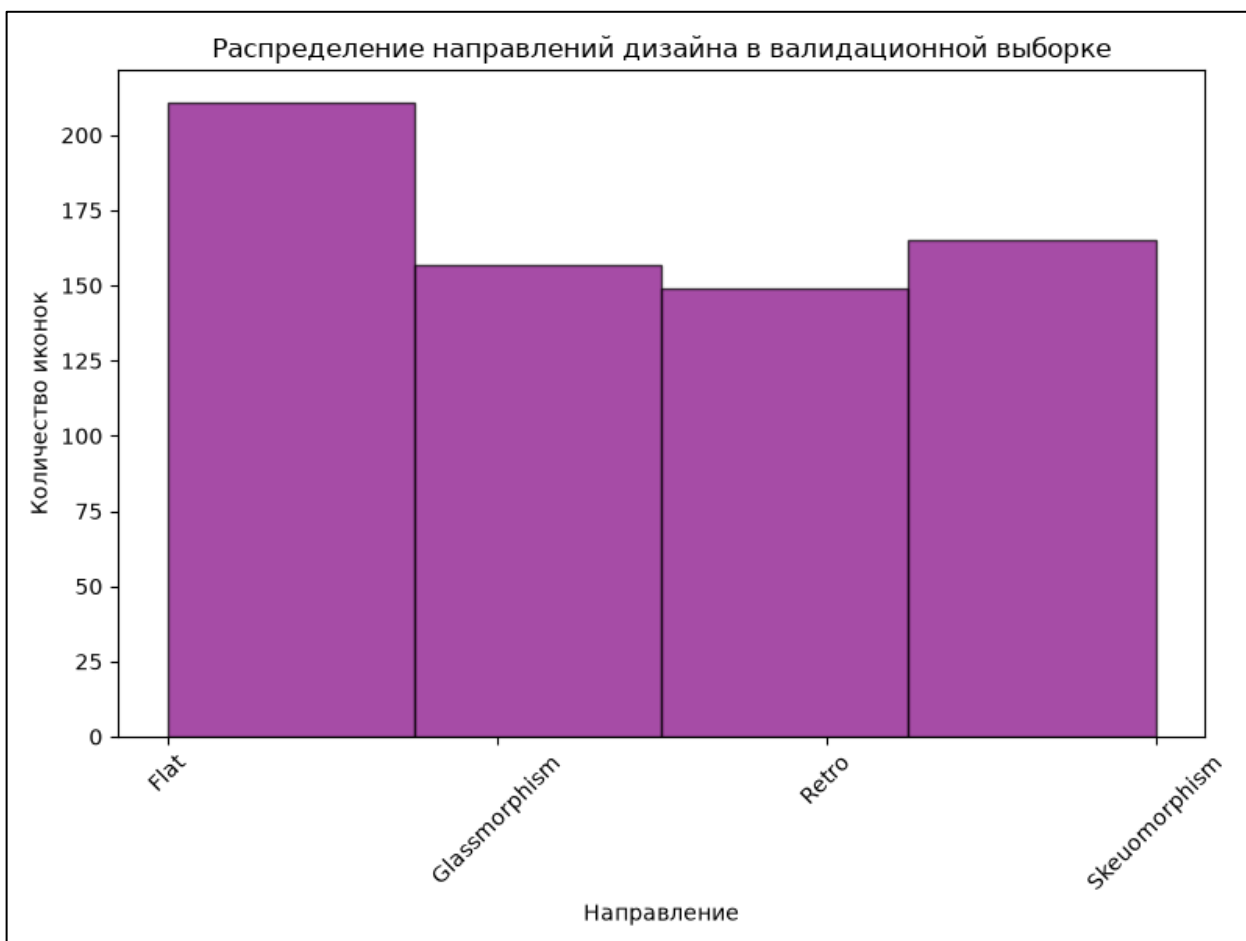


Рисунок 6.2 – распределение экземпляров классов в валидационной выборке для классификатора направлений дизайна

В процессе обучения производилось сохранение лучшей модели по критерию точности на валидационной выборке. Динамика изменения точности и значения функции потерь в процессе обучения представлены на рисунках 6.3 и 6.5 соответственно.

```

PS A:\Projects\Python_NSvOIIT> python neuronka.py

Устройство: cuda
Загружено иконок: 768
Направления дизайна: ['Flat', 'Glassmorphism', 'Retro', 'Skeuomorphism']

Распределение по направлениям:
  Flat: 369 иконок
  Glassmorphism: 87 иконок
  Skeuomorphism: 254 иконок
  Retro: 58 иконок

Максимальное количество иконок в одном направлении: 369
Целевое количество для каждого направления (x2): 738

Flat: 369 -> 738 (+369 аугментаций)

Glassmorphism: 87 -> 738 (+651 аугментаций)

Skeuomorphism: 254 -> 738 (+484 аугментаций)

Retro: 58 -> 738 (+680 аугментаций)

Train (направления): 2724 иконок
Val (направления): 682 иконок

Распределение по направлениям в Train (после балансировки):
  Flat: 896
  Glassmorphism: 626
  Retro: 605
  Skeuomorphism: 597
Гистограмма направлений сохранена: direction_distribution.png

Обучение модели для определения направления дизайна...
Epoch 1/30 | Loss: 1.1392 | Val Acc: 47.36% - Новый лучший
Epoch 2/30 | Loss: 0.7143 | Val Acc: 69.65% - Новый лучший
Epoch 3/30 | Loss: 0.5321 | Val Acc: 82.26% - Новый лучший
Epoch 4/30 | Loss: 0.3532 | Val Acc: 79.33% (лучший: 82.26%)
Epoch 5/30 | Loss: 0.2462 | Val Acc: 79.47% (лучший: 82.26%)
Epoch 6/30 | Loss: 0.2064 | Val Acc: 77.57% (лучший: 82.26%)
Epoch 7/30 | Loss: 0.2186 | Val Acc: 80.65% (лучший: 82.26%)
Epoch 8/30 | Loss: 0.2428 | Val Acc: 84.02% - Новый лучший
Epoch 9/30 | Loss: 0.1695 | Val Acc: 81.67% (лучший: 84.02%)
Epoch 10/30 | Loss: 0.1605 | Val Acc: 81.09% (лучший: 84.02%)
Epoch 11/30 | Loss: 0.0981 | Val Acc: 85.04% - Новый лучший
Epoch 12/30 | Loss: 0.0891 | Val Acc: 83.43% (лучший: 85.04%)
Epoch 13/30 | Loss: 0.0579 | Val Acc: 82.55% (лучший: 85.04%)
Epoch 14/30 | Loss: 0.1068 | Val Acc: 85.63% - Новый лучший
Epoch 15/30 | Loss: 0.0910 | Val Acc: 85.04% (лучший: 85.63%)
Epoch 16/30 | Loss: 0.0611 | Val Acc: 86.66% - Новый лучший
Epoch 17/30 | Loss: 0.0418 | Val Acc: 86.66% (лучший: 86.66%)
Epoch 18/30 | Loss: 0.0245 | Val Acc: 84.90% (лучший: 86.66%)
Epoch 19/30 | Loss: 0.0152 | Val Acc: 85.78% (лучший: 86.66%)
Epoch 20/30 | Loss: 0.0169 | Val Acc: 87.39% - Новый лучший
Epoch 21/30 | Loss: 0.0243 | Val Acc: 87.98% - Новый лучший
Epoch 22/30 | Loss: 0.0292 | Val Acc: 88.27% - Новый лучший
Epoch 23/30 | Loss: 0.0271 | Val Acc: 81.23% (лучший: 88.27%)
Epoch 24/30 | Loss: 0.0253 | Val Acc: 85.78% (лучший: 88.27%)
Epoch 25/30 | Loss: 0.0332 | Val Acc: 86.51% (лучший: 88.27%)
Epoch 26/30 | Loss: 0.0712 | Val Acc: 78.30% (лучший: 88.27%)
Epoch 27/30 | Loss: 0.1443 | Val Acc: 81.52% (лучший: 88.27%)
Epoch 28/30 | Loss: 0.1110 | Val Acc: 81.38% (лучший: 88.27%)
Epoch 29/30 | Loss: 0.0681 | Val Acc: 86.22% (лучший: 88.27%)

Early stopping на эпохе 29

```

Рисунок 6.3 – процесс обучения классификатора направлений дизайна



Рисунок 6.5 – график потерь для классификатора направлений дизайна

Анализ ошибок классификации проводился с помощью матрицы неточностей, которая позволяет визуальнo оценить, какие классы модель путает между собой. Матрица неточностей для классификатора направлений дизайна представлена на рисунке 6.4.

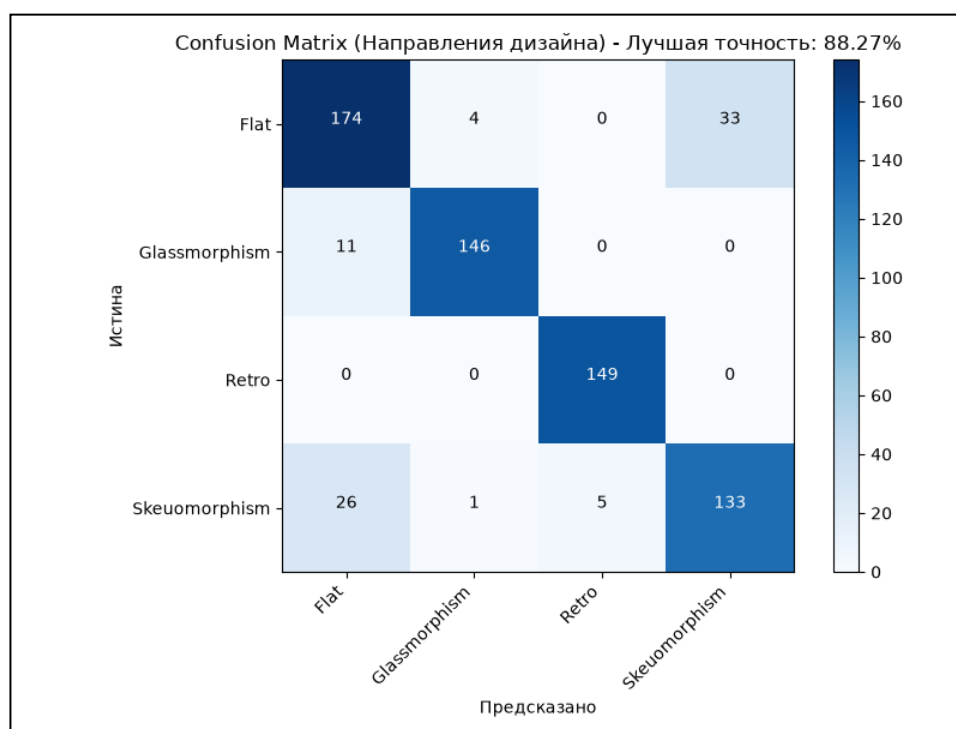


Рисунок 6.4 – матрица неточностей для классификатора направлений дизайна

Для классификатора стилей также была проведена балансировка классов до целевого значения 628 иконок на каждый из семи стилей. Распределение экземпляров классов в валидационной выборке представлено на рисунке 6.6.

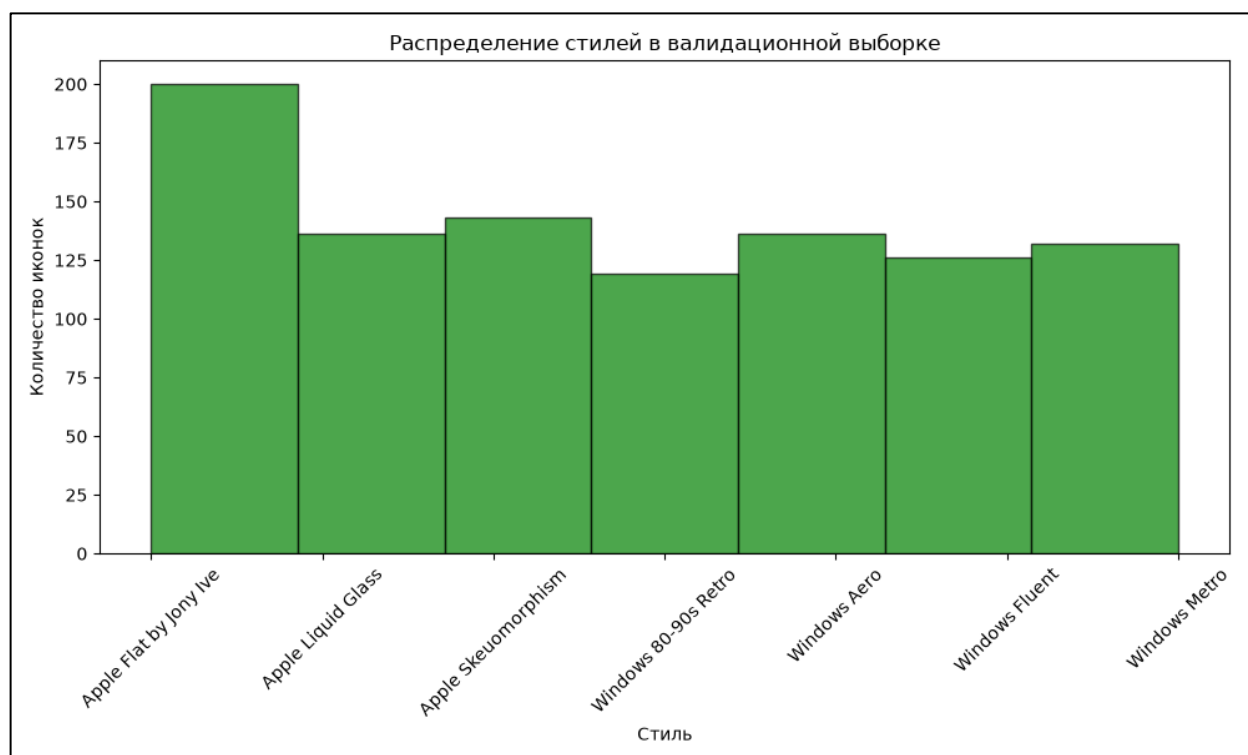


Рисунок 6.6 – распределение экземпляров классов в валидационной выборке для классификатора стилей

Процесс обучения классификатора стилей, включая динамику точности, представлен на рисунке 6.7. Значение функции потерь в процессе обучения приведено на рисунке 6.9.

```
Распределение по стилям:
  Apple Flat by Jony Ive: 314 иконок
  Apple Liquid Glass: 87 иконок
  Apple Skeuomorphism: 144 иконок
  Windows Aero: 110 иконок
  Windows Metro: 39 иконок
  Windows Fluent: 16 иконок
  Windows 80-90s Retro: 58 иконок

Максимальное количество иконок в одном стиле: 314
Целевое количество для каждого стиля (x2): 628

Apple Flat by Jony Ive: 314 -> 628 (+314 аугментаций)
Apple Liquid Glass: 87 -> 628 (+541 аугментаций)
Apple Skeuomorphism: 144 -> 628 (+484 аугментаций)
Windows Aero: 110 -> 628 (+518 аугментаций)
Windows Metro: 39 -> 628 (+589 аугментаций)
Windows Fluent: 16 -> 628 (+612 аугментаций)
Windows 80-90s Retro: 58 -> 628 (+570 аугментаций)

Train (стили): 3967 иконок
Val (стили): 992 иконок

Распределение по стилям в Train (после балансировки):
  Apple Flat by Jony Ive: 742
  Apple Liquid Glass: 560
  Apple Skeuomorphism: 577
  Windows 80-90s Retro: 519
  Windows Aero: 524
  Windows Fluent: 514
  Windows Metro: 531
Гистограмма стилей сохранена: style_distribution.png

Обучение модели для определения стиля...
Количество классов стилей: 7
Epoch 1/30 | Loss: 1.5203 | Val Acc: 48.08% - НОВЫЙ ЛУЧШИЙ
Epoch 2/30 | Loss: 0.8153 | Val Acc: 68.65% - НОВЫЙ ЛУЧШИЙ
Epoch 3/30 | Loss: 0.5300 | Val Acc: 73.19% - НОВЫЙ ЛУЧШИЙ
Epoch 4/30 | Loss: 0.3986 | Val Acc: 76.71% - НОВЫЙ ЛУЧШИЙ
Epoch 5/30 | Loss: 0.2673 | Val Acc: 73.79% (лучший: 76.71%)
Epoch 6/30 | Loss: 0.2632 | Val Acc: 78.63% - НОВЫЙ ЛУЧШИЙ
Epoch 7/30 | Loss: 0.2031 | Val Acc: 80.75% - НОВЫЙ ЛУЧШИЙ
Epoch 8/30 | Loss: 0.1848 | Val Acc: 80.95% - НОВЫЙ ЛУЧШИЙ
Epoch 9/30 | Loss: 0.1711 | Val Acc: 84.98% - НОВЫЙ ЛУЧШИЙ
Epoch 10/30 | Loss: 0.1116 | Val Acc: 84.17% (лучший: 84.98%)
Epoch 11/30 | Loss: 0.1372 | Val Acc: 81.96% (лучший: 84.98%)
Epoch 12/30 | Loss: 0.0921 | Val Acc: 82.26% (лучший: 84.98%)
Epoch 13/30 | Loss: 0.0542 | Val Acc: 83.67% (лучший: 84.98%)
Epoch 14/30 | Loss: 0.0246 | Val Acc: 88.51% - НОВЫЙ ЛУЧШИЙ
Epoch 15/30 | Loss: 0.0115 | Val Acc: 89.11% - НОВЫЙ ЛУЧШИЙ
Epoch 16/30 | Loss: 0.0053 | Val Acc: 89.01% (лучший: 89.11%)
Epoch 17/30 | Loss: 0.0041 | Val Acc: 89.42% - НОВЫЙ ЛУЧШИЙ
Epoch 18/30 | Loss: 0.0021 | Val Acc: 90.02% - НОВЫЙ ЛУЧШИЙ
Epoch 19/30 | Loss: 0.0017 | Val Acc: 88.71% (лучший: 90.02%)
Epoch 20/30 | Loss: 0.0015 | Val Acc: 88.31% (лучший: 90.02%)
Epoch 21/30 | Loss: 0.0015 | Val Acc: 88.41% (лучший: 90.02%)
Epoch 22/30 | Loss: 0.0014 | Val Acc: 88.91% (лучший: 90.02%)
Epoch 23/30 | Loss: 0.0009 | Val Acc: 88.61% (лучший: 90.02%)

Ранняя остановка на эпохе 23
```

Рисунок 6.7 – процесс обучения классификатора стилей

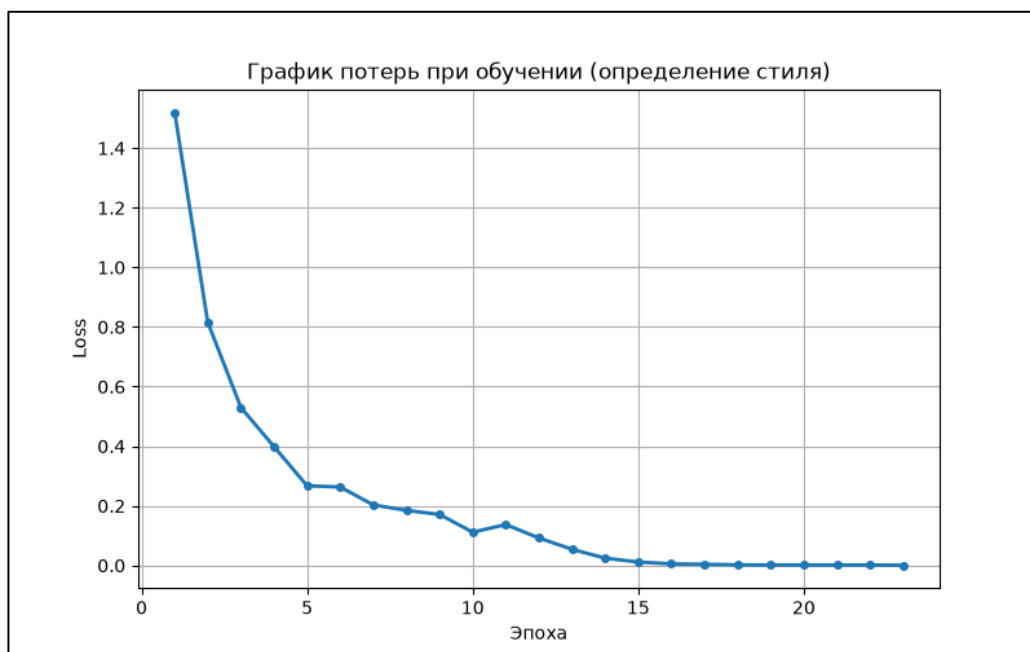


Рисунок 6.5 – график потерь для классификатора стилей

На рисунке 6.8 представлена матрица неточностей для классификатора стилей, позволяющая выявить наиболее проблемные пары классов.

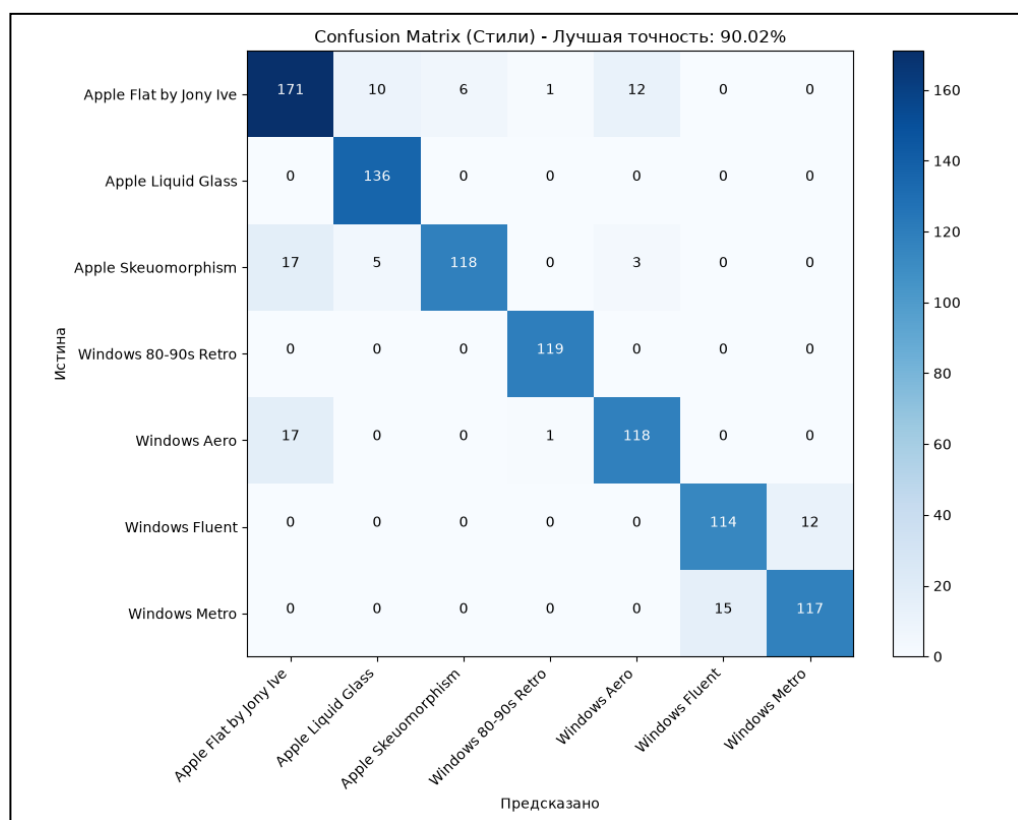


Рисунок 6.8 – матрица неточностей для классификатора стилей

Классификатор операционных систем обучался на 19 классах, соответствующих различным версиям Windows, macOS и iOS. Распределение экземпляров классов в валидационной выборке представлено на рисунке 6.10.

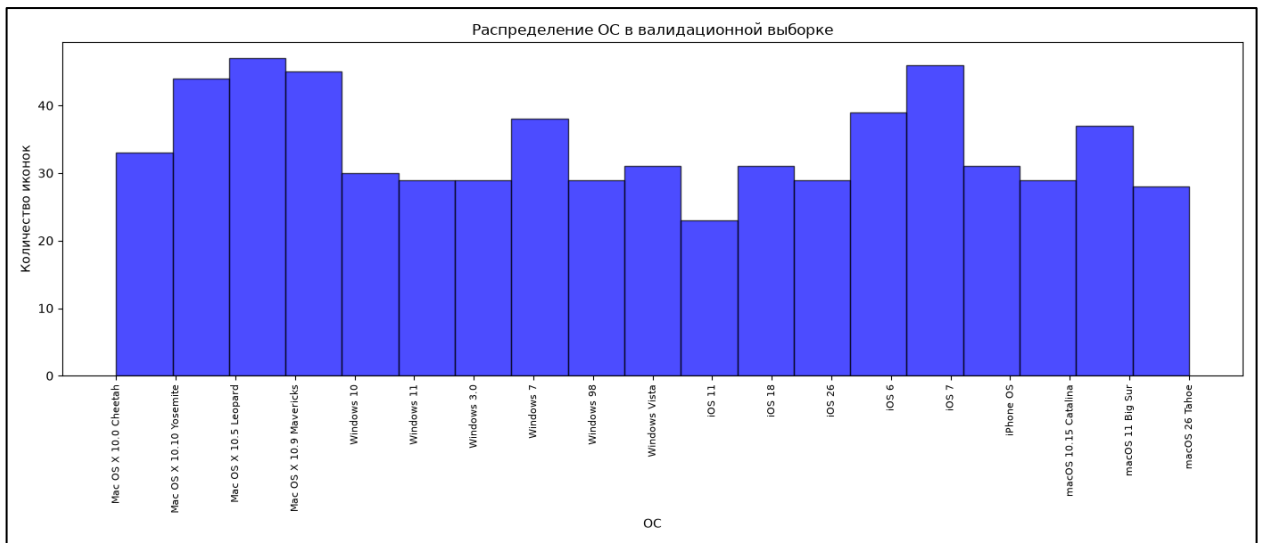


Рисунок 6.10 – распределение экземпляров классов в валидационной выборке для классификатора ОС

Процесс обучения классификатора ОС представлен на рисунке 6.11, график функции потерь – на рисунке 6.13.

```
macOS 10.15 Catalina: 76 -> 152 (+76 аугментаций)
macOS 11 Big Sur: 63 -> 152 (+89 аугментаций)
macOS 26 Tahoe: 52 -> 152 (+100 аугментаций)
Windows 10: 39 -> 152 (+113 аугментаций)
Windows 11: 16 -> 152 (+136 аугментаций)
Windows 3.0: 17 -> 152 (+135 аугментаций)
Windows 7: 26 -> 152 (+126 аугментаций)
Windows 98: 41 -> 152 (+111 аугментаций)
Windows Vista: 51 -> 152 (+101 аугментаций)

Train (OC): 2588 иконок
Val (OC): 648 иконок

Распределение по ОС в Train (после балансировки):
  Mac OS X 10.0 Cheetah: 123
  Mac OS X 10.10 Yosemite: 148
  Mac OS X 10.5 Leopard: 151
  Mac OS X 10.9 Mavericks: 145
  Windows 10: 126
  Windows 11: 131
  Windows 3.0: 124
  Windows 7: 118
  Windows 98: 135
  Windows Vista: 122
  iOS 11: 137
  iOS 18: 134
  iOS 26: 146
  iOS 6: 121
  iOS 7: 119
  iPhone OS: 129
  macOS 10.15 Catalina: 199
  macOS 11 Big Sur: 152
  macOS 26 Tahoe: 128

Гистограмма ОС сохранена: os_distribution.png

Обучение модели для определения ОС...
Количество классов ОС: 19
Epoch 1/30 | Loss: 2.3433 | Val Acc: 29.48% – НОВЫЙ ЛУЧШИЙ
Epoch 2/30 | Loss: 1.5246 | Val Acc: 51.23% – НОВЫЙ ЛУЧШИЙ
Epoch 3/30 | Loss: 0.9764 | Val Acc: 54.32% – НОВЫЙ ЛУЧШИЙ
Epoch 4/30 | Loss: 0.5989 | Val Acc: 59.72% – НОВЫЙ ЛУЧШИЙ
Epoch 5/30 | Loss: 0.4319 | Val Acc: 60.80% – НОВЫЙ ЛУЧШИЙ
Epoch 6/30 | Loss: 0.2897 | Val Acc: 66.51% – НОВЫЙ ЛУЧШИЙ
Epoch 7/30 | Loss: 0.2432 | Val Acc: 59.41% (лучший: 66.51%)
Epoch 8/30 | Loss: 0.2272 | Val Acc: 59.72% (лучший: 66.51%)
Epoch 9/30 | Loss: 0.2329 | Val Acc: 61.27% (лучший: 66.51%)
Epoch 10/30 | Loss: 0.1838 | Val Acc: 62.65% (лучший: 66.51%)
Epoch 11/30 | Loss: 0.0912 | Val Acc: 72.38% – НОВЫЙ ЛУЧШИЙ
Epoch 12/30 | Loss: 0.0279 | Val Acc: 72.84% – НОВЫЙ ЛУЧШИЙ
Epoch 13/30 | Loss: 0.0250 | Val Acc: 71.91% (лучший: 72.84%)
Epoch 14/30 | Loss: 0.0127 | Val Acc: 71.30% (лучший: 72.84%)
Epoch 15/30 | Loss: 0.0077 | Val Acc: 71.91% (лучший: 72.84%)
Epoch 16/30 | Loss: 0.0050 | Val Acc: 72.84% (лучший: 72.84%)
Epoch 17/30 | Loss: 0.0034 | Val Acc: 72.07% (лучший: 72.84%)

Ранняя остановка на эпохе 17
```

Рисунок 6.11 – процесс обучения классификатора ОС

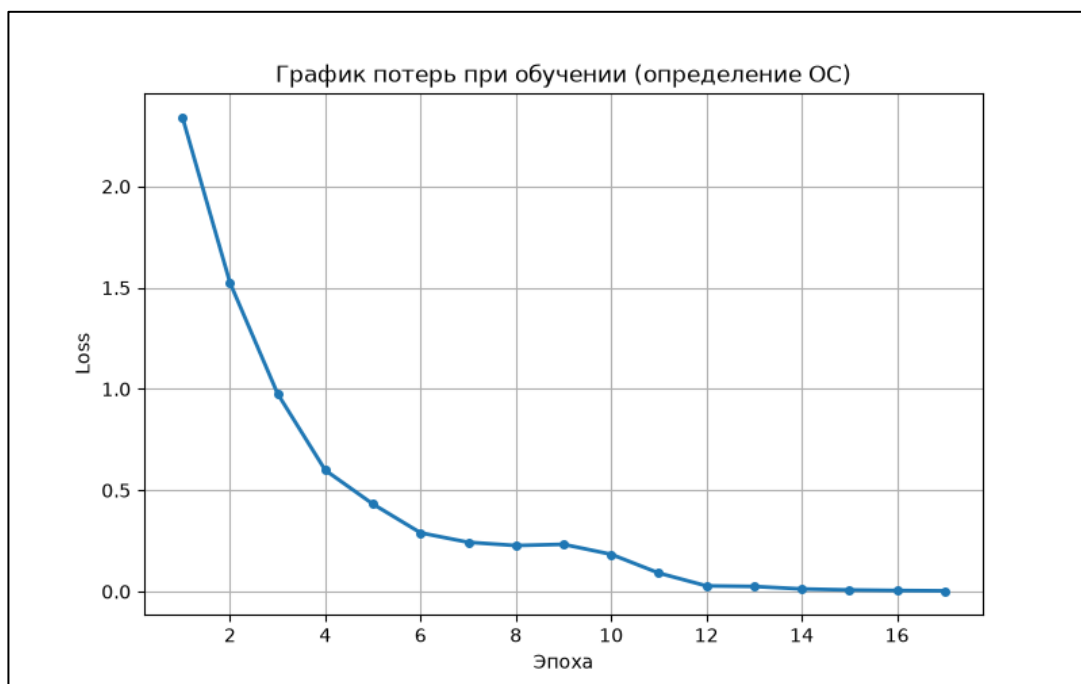


Рисунок 6.13 – график потерь для классификатора ОС

Матрица неточностей для классификатора ОС (рисунок 6.12) демонстрирует, какие версии операционных систем наиболее часто путаются между собой.

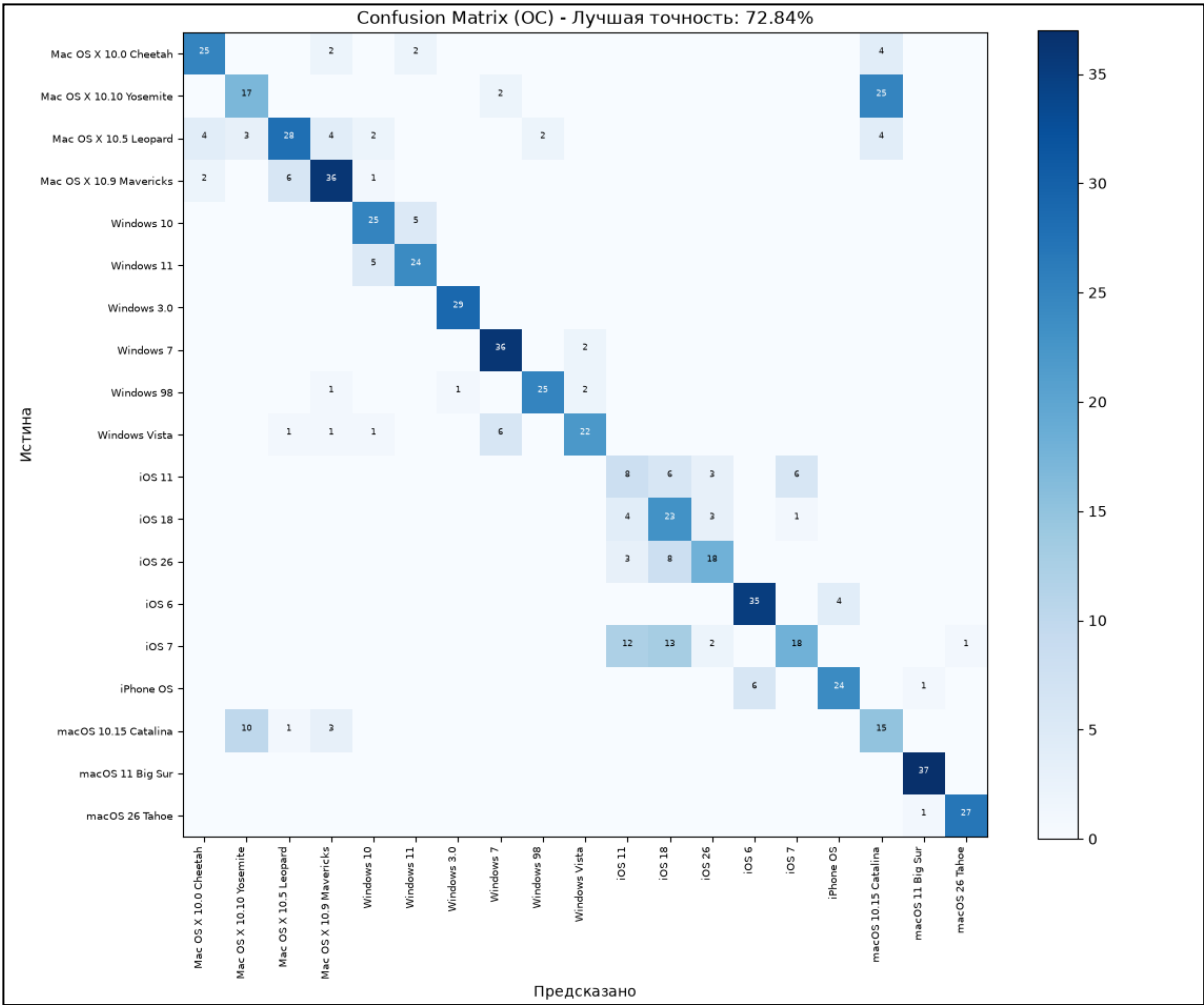


Рисунок 6.12 – матрица неточностей для классификатора ОС

После завершения обучения всех трёх классификаторов была проведена проверка на пользовательских значках, помещённых в папку User. Результаты анализа для значков Finder, iTunes и Mail представлены на рисунке 6.14.

```
Finder_converted.png
-----
Направление дизайна: Glassmorphism (уверенность 99.4%)
Стиль: Apple Liquid Glass (уверенность 100.0%)
ОС: iOS 26 (уверенность 77.3%)

Детализация по направлениям:
  1. Glassmorphism: 99.4%
  2. Flat: 0.5%
  3. Skeuomorphism: 0.0%
  4. Retro: 0.0%

Детализация по стилям (топ-5):
  1. Apple Liquid Glass: 100.0%

Детализация по ОС (топ-5):
  1. iOS 26: 77.3%
  2. macOS 26 Tahoe: 22.7%

iTunes_converted.png
-----
Направление дизайна: Skeuomorphism (уверенность 99.3%)
Стиль: Apple Skeuomorphism (уверенность 61.1%)
ОС: Mac OS X 10.5 Leopard (уверенность 85.3%)

Детализация по направлениям:
  1. Skeuomorphism: 99.3%
  2. Retro: 0.4%
  3. Glassmorphism: 0.2%
  4. Flat: 0.1%

Детализация по стилям (топ-5):
  1. Apple Skeuomorphism: 61.1%
  2. Windows Aero: 38.9%

Детализация по ОС (топ-5):
  1. Mac OS X 10.5 Leopard: 85.3%
  2. iOS 6: 9.6%
  3. Mac OS X 10.0 Cheetah: 4.8%
  4. Mac OS X 10.9 Mavericks: 0.3%
  5. iPhone OS: 0.1%

Mail_converted.png
-----
Направление дизайна: Skeuomorphism (уверенность 97.7%)
Стиль: Windows Aero (уверенность 66.4%)
ОС: Windows Vista (уверенность 95.2%)

Детализация по направлениям:
  1. Skeuomorphism: 97.7%
  2. Retro: 2.1%
  3. Flat: 0.2%
  4. Glassmorphism: 0.0%

Детализация по стилям (топ-5):
  1. Windows Aero: 66.4%
  2. Apple Skeuomorphism: 33.6%

Детализация по ОС (топ-5):
  1. Windows Vista: 95.2%
  2. Windows 7: 4.8%
```

Рисунок 6.14 – результаты анализа Finder, iTunes и Mail

Результаты анализа для значков News, Notes и Photos представлены на рисунке 6.15.

```
News_converted.png
-----
Направление дизайна: Glassmorphism (уверенность 91.7%)
Стиль: Apple Liquid Glass (уверенность 100.0%)
ОС: iOS 26 (уверенность 91.7%)

Детализация по направлениям:
1. Glassmorphism: 91.7%
2. Flat: 8.3%
3. Skeuomorphism: 0.0%
4. Retro: 0.0%

Детализация по стилям (топ-5):
1. Apple Liquid Glass: 100.0%

Детализация по ОС (топ-5):
1. iOS 26: 91.7%
2. macOS 26 Tahoe: 8.3%

Notes_converted.png
-----
Направление дизайна: Flat (уверенность 86.9%)
Стиль: Apple Flat by Jony Ive (уверенность 100.0%)
ОС: iOS 11 (уверенность 93.0%)

Детализация по направлениям:
1. Flat: 86.9%
2. Glassmorphism: 12.8%
3. Skeuomorphism: 0.3%
4. Retro: 0.0%

Детализация по стилям (топ-5):
1. Apple Flat by Jony Ive: 100.0%
2. Windows Fluent: 0.0%
3. Windows Metro: 0.0%

Детализация по ОС (топ-5):
1. iOS 11: 93.0%
2. iOS 7: 4.6%
3. iOS 18: 2.2%
4. macOS 11 Big Sur: 0.1%
5. macOS 10.15 Catalina: 0.0%

Photos_converted.png
-----
Направление дизайна: Flat (уверенность 92.4%)
Стиль: Apple Flat by Jony Ive (уверенность 96.7%)
ОС: iOS 11 (уверенность 69.6%)

Детализация по направлениям:
1. Flat: 92.4%
2. Glassmorphism: 7.2%
3. Skeuomorphism: 0.4%
4. Retro: 0.1%

Детализация по стилям (топ-5):
1. Apple Flat by Jony Ive: 96.7%
2. Windows Fluent: 2.7%
3. Windows Metro: 0.6%

Детализация по ОС (топ-5):
1. iOS 11: 69.6%
2. iOS 18: 22.8%
3. macOS 11 Big Sur: 6.2%
4. iOS 7: 1.1%
5. Mac OS X 10.10 Yosemite: 0.2%
```

Рисунок 6.15 – результаты анализа News, Notes и Photos

Из результатов анализа видно, что модель отлично различает направления дизайна Flat и Skeuomorphism. Однако, уже с трудом распознаёт Flat и Glassmorphism. Это вызвано тем, что представленные в датасете значки в направлении глассморфизма принадлежали корпорации Apple и по большей части повторяли те же версии значков, только в плоском исполнении[3]. Помимо этого, модель слабо распознаёт ОС и направление стиля, путая продукты Apple с продуктами Microsoft. В самом коде модели не прописана полностью логическая цепочка, позволяющая выбирать конкретную ОС, основываясь на полных результатах анализа направления дизайна и стиля. Также одной из причин ошибок является изначальное низкое разрешение значков. Они были извлечены в разрешении 32 на 32 пикселя. Увеличение до 128 на 128 пикселей и стандартизация в формат PNG не улучшают качество обучения.

Заключение

В ходе выполнения курсовой работы была разработана система классификации стандартных значков операционных систем, для которой был собран размеченный набор данных, включающий более 700 иконок, распределённых по 19 версиям Windows, macOS и iOS. Проведена предварительная обработка изображений: приведение к единому формату, нормализация, балансировка классов с помощью аугментации.

Разработана и обучена каскадная нейросетевая система на основе архитектуры ResNet18, состоящая из трёх последовательных классификаторов: для определения направления дизайна (4 класса), стиля (7 классов) и версии операционной системы (19 классов). Точность классификации направления дизайна на валидационной выборке составила 88%. Анализ ошибок выявил путаницу между визуально близкими направлениями Flat и Glassmorphism.

По проделанной работе был написан отчёт согласно требованиям ОС ТУСУР[1].

Исходный код проекта размещен на репозитории GitHub:

<https://github.com/timursuleimanov2007-ship-it/nsvoiit.git>

Список использованных источников

1. Образовательный стандарт вуза ОС ТУСУР 01-2021 [Электронный ресурс]: сайт ТУСУРа. URL: <https://regulations.tusur.ru/#/documents/87> (дата обращения: 15.06.2026)
2. GUIs [Электронный ресурс]: // GUIs: сайт. – URL: <https://guis.org/> (дата обращения: 15.06.2026)
3. Описание обновлений для macOS Tahoe 26 [Электронный ресурс]: // support.apple.com: сайт. – URL: <https://support.apple.com/ru-ru/122868> (дата обращения: 15.06.2026)
4. Что такое дизайн: понятия, разновидности [Электронный ресурс]: // gb.ru: сайт. – URL: <https://gb.ru/blog/chto-takoe-dizajn/> (дата обращения: 15.06.2026)
5. Python 3.11.11 документация [Электронный ресурс]: // docs.python: сайт. – URL: <https://docs.python.org/3.11/> (дата обращения: 15.06.2026)
6. PyTorch [Электронный ресурс]: // PyTorch: сайт. – URL: <https://pytorch.org/> (дата обращения: 15.06.2026)

ПРИЛОЖЕНИЕ А

Листинг кода

```
import os

import json

import glob

import shutil

from pathlib import Path

from PIL import Image

import numpy as np

import matplotlib.pyplot as plt

from sklearn.metrics import accuracy_score, classification_report,
confusion_matrix

from sklearn.model_selection import train_test_split

import torch

import torch.nn as nn

import torch.nn.functional as F

import torch.optim as optim

from torch.utils.data import Dataset, DataLoader, random_split

from torchvision import transforms, models

from torchvision.utils import make_grid

import random

import time

from datetime import datetime
```



```

def user():

    base_path = os.path.dirname(__file__)

    user_folder = os.path.join(base_path, "User")

    if not os.path.exists(user_folder):

        os.makedirs(user_folder)

        print(f"Создана папка: {user_folder}")

        print("Положите туда иконки и запустите снова")

        return [], []

    supported_extensions = {'.png', '.jpg', '.jpeg', '.ico', '.svg', '.icns', '.gif', '.bmp', '.tiff',
                             '.webp'}

    converted_folder = os.path.join(user_folder, "converted")

    os.makedirs(converted_folder, exist_ok=True)

    image_files = []

    for filename in os.listdir(user_folder):

        file_path = os.path.join(user_folder, filename)

        if os.path.isdir(file_path):

```

```

        continue

    ext = os.path.splitext(filename)[1].lower()

    if ext not in supported_extensions:

        continue

    image_files.append(file_path)

if not image_files:

    print("В папке User не найдено изображений")

    print(f"Поддерживаются: {supported_extensions}")

    return [], []

print(f"Найдено изображений: {len(image_files)}")

images = []

image_paths = []

for img_path in image_files:

    try:

        with Image.open(img_path) as img:

            if img.mode in ('RGBA', 'LA', 'P'):

```

```

background = Image.new("RGB", img.size, (255, 255, 255))

if img.mode == 'P':

    img = img.convert('RGBA')

    background.paste(img, mask=img.split()[-1] if img.mode == 'RGBA'
else None)

    img = background

elif img.mode != 'RGB':

    img = img.convert('RGB')


target_size = 128

img_w, img_h = img.size

scale = min(target_size / img_w, target_size / img_h)

new_w = int(img_w * scale)

new_h = int(img_h * scale)


img_resized = img.resize((new_w, new_h),
Image.Resampling.LANCZOS)


new_img = Image.new("RGB", (target_size, target_size), (255, 255, 255))

x_offset = (target_size - new_w) // 2

y_offset = (target_size - new_h) // 2

new_img.paste(img_resized, (x_offset, y_offset))

```

```

name_without_ext = os.path.splitext(os.path.basename(img_path))[0]

output_filename = f'{name_without_ext}_converted.png'

output_path = os.path.join(converted_folder, output_filename)

new_img.save(output_path, "PNG")


images.append(new_img)

image_paths.append(output_path)


print(f'{os.path.basename(img_path)} → {output_filename}')


except Exception as e:

    print(f'Ошибка {os.path.basename(img_path)}: {e}')


print(f'\nКонвертировано {len(images)} иконок в {converted_folder}')


return images, image_paths


def loadDirection(labels_file, root_dir, batch_size=32, train_ratio=0.8,
image_size=128):

    with open(labels_file, 'r', encoding='utf-8') as f:

        labels = json.load(f)

```

```

direction_to_idx = {direction: i for i, direction in
enumerate(sorted(set(item["direction"] for item in labels)))}

idx_to_direction = {i: direction for direction, i in direction_to_idx.items()}

num_directions = len(direction_to_idx)

print(f'Загружено иконок: {len(labels)}')

print(f'Направления дизайна: {list(direction_to_idx.keys())}')


direction_groups = {}

for item in labels:

    direction = item["direction"]

    if direction not in direction_groups:

        direction_groups[direction] = []

    direction_groups[direction].append(item)


print(f'\nРаспределение по направлениям:')

for direction, items in direction_groups.items():

    print(f' {direction}: {len(items)} иконок")


max_count = max(len(items) for items in direction_groups.values())

target_count = max_count * 2

print(f'\nМаксимальное количество иконок в одном направлении:
{max_count}')

```

```

print(f'Целевое количество для каждого направления (x2): {target_count}')

def get_heavy_augmentation():

    return transforms.Compose([

        transforms.Resize((image_size, image_size)),

        transforms.RandomHorizontalFlip(p=0.5),

        transforms.RandomRotation(degrees=25),

        transforms.ColorJitter(brightness=0.4,    contrast=0.4,    saturation=0.4,
hue=0.15),

        transforms.RandomAffine(degrees=15,  translate=(0.2, 0.2),  scale=(0.8,
1.2)),

        transforms.GaussianBlur(kernel_size=(3, 3), sigma=(0.1, 1.0)),

        transforms.ToTensor(),

        transforms.Normalize(mean=[0.485, 0.456, 0.406],

                                std=[0.229, 0.224, 0.225])

    ])

def get_light_augmentation():

    return transforms.Compose([

        transforms.Resize((image_size, image_size)),

        transforms.RandomHorizontalFlip(p=0.3),

        transforms.RandomRotation(degrees=10),

        transforms.ColorJitter(brightness=0.15, contrast=0.15, saturation=0.15),

```

```

transforms.ToTensor(),

transforms.Normalize(mean=[0.485, 0.456, 0.406],

                      std=[0.229, 0.224, 0.225])

])

```

```

val_transform = transforms.Compose([

    transforms.Resize((image_size, image_size)),

    transforms.ToTensor(),

    transforms.Normalize(mean=[0.485, 0.456, 0.406],

                        std=[0.229, 0.224, 0.225])

])

```

```

balanced_items = []

```

```

for direction, items in direction_groups.items():

    current_count = len(items)

    if current_count < target_count:

        copies_needed = target_count - current_count

        copies_per_item = copies_needed // current_count + 1

        print(f"\n{direction}: {current_count} -> {target_count} (+{copies_needed}
аугментаций)")

```

```

    for item in items:

        balanced_items.append((item, False))

        for _ in range(copies_per_item):

            balanced_items.append((item, True))

    else:

        print(f"\n{direction}: {current_count} -> {target_count} (без изменений)")

        for item in items:

            balanced_items.append((item, False))


random.shuffle(balanced_items)


def get_item(item_data, is_val=False):

    item, is_augmented = item_data

    img_path = os.path.join(root_dir, item["file"])

    try:

        image = Image.open(img_path).convert("RGB")

    except Exception:

        image = Image.new("RGB", (image_size, image_size), (0, 0, 0))

    if is_val:

        transform = val_transform

```



```
elif is_augmented:

    transform = get_heavy_augmentation()

else:

    transform = get_light_augmentation()

image = transform(image)

direction_label = direction_to_idx[item["direction"]]

return image, direction_label
```

```
train_size = int(train_ratio * len(balanced_items))
```

```
val_size = len(balanced_items) - train_size
```

```
train_items = balanced_items[:train_size]
```

```
val_items = balanced_items[train_size:]
```

```
train_data = [get_item(item, is_val=False) for item in train_items]
```

```
val_data = [get_item(item, is_val=True) for item in val_items]
```

```
train_loader = DataLoader(train_data, batch_size=batch_size, shuffle=True)
```

```
val_loader = DataLoader(val_data, batch_size=batch_size, shuffle=False)
```

```

print(f'\nTrain (направления): {len(train_data)} иконок")

print(f'\nVal (направления): {len(val_data)} иконок")


train_labels = [label for _, label in train_data]

label_counts = {}

for s in train_labels:

    name = idx_to_direction[s]

    label_counts[name] = label_counts.get(name, 0) + 1


print(f'\nРаспределение по направлениям в Train (после балансировки):")

for name, count in sorted(label_counts.items()):

    print(f' {name}: {count}")


return train_loader, val_loader, num_directions, idx_to_direction


def loadStyle(labels_file, root_dir, batch_size=32, train_ratio=0.8,
image_size=128):

    with open(labels_file, 'r', encoding='utf-8') as f:

        labels = json.load(f)


    style_to_idx = {style: i for i, style in enumerate(sorted(set(item["style"] for item
in labels)))}

    idx_to_style = {i: style for style, i in style_to_idx.items()}

```

```

num_styles = len(style_to_idx)

print(f"Загружено иконок: {len(labels)}")

print(f"Стили: {list(style_to_idx.keys())}")

style_groups = {}

for item in labels:

    style = item["style"]

    if style not in style_groups:

        style_groups[style] = []

    style_groups[style].append(item)

print(f"\nРаспределение по стилям:")

for style, items in style_groups.items():

    print(f" {style}: {len(items)} иконок")

max_count = max(len(items) for items in style_groups.values())

target_count = max_count * 2

print(f"\nМаксимальное количество иконок в одном стиле: {max_count}")

print(f"Целевое количество для каждого стиля (x2): {target_count}")

def get_heavy_augmentation():

```

```

return transforms.Compose([

    transforms.Resize((image_size, image_size)),

    transforms.RandomHorizontalFlip(p=0.5),

    transforms.RandomRotation(degrees=25),

    transforms.ColorJitter(brightness=0.4,    contrast=0.4,    saturation=0.4,
hue=0.15),

    transforms.RandomAffine(degrees=15,  translate=(0.2, 0.2),  scale=(0.8,
1.2)),

    transforms.GaussianBlur(kernel_size=(3, 3), sigma=(0.1, 1.0)),

    transforms.ToTensor(),

    transforms.Normalize(mean=[0.485, 0.456, 0.406],
                        std=[0.229, 0.224, 0.225])

])

```

```

def get_light_augmentation():

```

```

    return transforms.Compose([

        transforms.Resize((image_size, image_size)),

        transforms.RandomHorizontalFlip(p=0.3),

        transforms.RandomRotation(degrees=10),

        transforms.ColorJitter(brightness=0.15, contrast=0.15, saturation=0.15),

        transforms.ToTensor(),

        transforms.Normalize(mean=[0.485, 0.456, 0.406],
                        std=[0.229, 0.224, 0.225])
    ])

```

```
)
```

```
val_transform = transforms.Compose([  
    transforms.Resize((image_size, image_size)),  
    transforms.ToTensor(),  
    transforms.Normalize(mean=[0.485, 0.456, 0.406],  
                          std=[0.229, 0.224, 0.225])
```

```
)
```

```
balanced_items = []
```

```
for style, items in style_groups.items():
```

```
    current_count = len(items)
```

```
    if current_count < target_count:
```

```
        copies_needed = target_count - current_count
```

```
        copies_per_item = copies_needed // current_count + 1
```

```
        print(f"\n{style}: {current_count} -> {target_count} (+{copies_needed}  
аугментаций)")
```

```
    for item in items:
```

```
        balanced_items.append((item, False))
```

```
    for _ in range(copies_per_item):
```

```

        balanced_items.append((item, True))

    else:

        print(f"\n{style}: {current_count} -> {target_count} (без изменений)")

        for item in items:

            balanced_items.append((item, False))


random.shuffle(balanced_items)


def get_item(item_data, is_val=False):

    item, is_augmented = item_data

    img_path = os.path.join(root_dir, item["file"])

    try:

        image = Image.open(img_path).convert("RGB")

    except Exception:

        image = Image.new("RGB", (image_size, image_size), (0, 0, 0))


    if is_val:

        transform = val_transform

    elif is_augmented:

        transform = get_heavy_augmentation()

    else:

```

```

transform = get_light_augmentation()

image = transform(image)

style_label = style_to_idx[item["style"]]

return image, style_label

train_size = int(train_ratio * len(balanced_items))
val_size = len(balanced_items) - train_size

train_items = balanced_items[:train_size]
val_items = balanced_items[train_size:]

train_data = [get_item(item, is_val=False) for item in train_items]
val_data = [get_item(item, is_val=True) for item in val_items]

train_loader = DataLoader(train_data, batch_size=batch_size, shuffle=True)
val_loader = DataLoader(val_data, batch_size=batch_size, shuffle=False)

print(f'\nTrain (стили): {len(train_data)} иконок')
print(f'\nVal (стили): {len(val_data)} иконок')

```

```

train_labels = [label for _, label in train_data]

label_counts = {}

for s in train_labels:

    name = idx_to_style[s]

    label_counts[name] = label_counts.get(name, 0) + 1


print(f'\nРаспределение по стилям в Train (после балансировки):')

for name, count in sorted(label_counts.items()):

    print(f' {name}: {count}')


return train_loader, val_loader, num_styles, idx_to_style


def loadOS(labels_file, root_dir, batch_size=32, train_ratio=0.8, image_size=128):

    with open(labels_file, 'r', encoding='utf-8') as f:

        labels = json.load(f)

        os_to_idx = {os_name: i for i, os_name in enumerate(sorted(set(item["os"] for
item in labels)))}

        idx_to_os = {i: os_name for os_name, i in os_to_idx.items()}

        num_os = len(os_to_idx)


    print(f'Загружено иконок: {len(labels)}')

    print(f'OC: {list(os_to_idx.keys())}')

```



```

os_groups = {}

for item in labels:

    os_name = item["os"]

    if os_name not in os_groups:

        os_groups[os_name] = []

    os_groups[os_name].append(item)


print(f"\nРаспределение по ОС:")

for os_name, items in os_groups.items():

    print(f" {os_name}: {len(items)} иконок")


max_count = max(len(items) for items in os_groups.values())

target_count = max_count * 2

print(f"\nМаксимальное количество иконок в одной ОС: {max_count}")

print(f"Целевое количество для каждой ОС (x2): {target_count}")


def get_heavy_augmentation():

    return transforms.Compose([

        transforms.Resize((image_size, image_size)),

        transforms.RandomHorizontalFlip(p=0.5),

        transforms.RandomRotation(degrees=25),

```

```

        transforms.ColorJitter(brightness=0.4,    contrast=0.4,    saturation=0.4,
hue=0.15),

        transforms.RandomAffine(degrees=15,  translate=(0.2, 0.2),  scale=(0.8,
1.2)),

        transforms.GaussianBlur(kernel_size=(3, 3), sigma=(0.1, 1.0)),

        transforms.ToTensor(),

        transforms.Normalize(mean=[0.485, 0.456, 0.406],

                                std=[0.229, 0.224, 0.225])

    ])

```

```

def get_light_augmentation():

```

```

    return transforms.Compose([

        transforms.Resize((image_size, image_size)),

        transforms.RandomHorizontalFlip(p=0.3),

        transforms.RandomRotation(degrees=10),

        transforms.ColorJitter(brightness=0.15, contrast=0.15, saturation=0.15),

        transforms.ToTensor(),

        transforms.Normalize(mean=[0.485, 0.456, 0.406],

                                std=[0.229, 0.224, 0.225])

    ])

```

```

val_transform = transforms.Compose([

    transforms.Resize((image_size, image_size)),

```

```

transforms.ToTensor(),

transforms.Normalize(mean=[0.485, 0.456, 0.406],
                      std=[0.229, 0.224, 0.225])

])

balanced_items = []

for os_name, items in os_groups.items():

    current_count = len(items)

    if current_count < target_count:

        copies_needed = target_count - current_count

        copies_per_item = copies_needed // current_count + 1

        print(f"\n{os_name}: {current_count} -> {target_count} (+{copies_needed}
аугментаций)")

        for item in items:

            balanced_items.append((item, False))

            for _ in range(copies_per_item):

                balanced_items.append((item, True))

        else:

            print(f"\n{os_name}: {current_count} -> {target_count} (без изменений)")

            for item in items:

```

```

        balanced_items.append((item, False))

random.shuffle(balanced_items)

def get_item(item_data, is_val=False):
    item, is_augmented = item_data

    img_path = os.path.join(root_dir, item["file"])

    try:
        image = Image.open(img_path).convert("RGB")
    except Exception:
        image = Image.new("RGB", (image_size, image_size), (0, 0, 0))

    if is_val:
        transform = val_transform
    elif is_augmented:
        transform = get_heavy_augmentation()
    else:
        transform = get_light_augmentation()

    image = transform(image)

    os_label = os_to_idx[item["os"]]

```

```
return image, os_label
```

```
train_size = int(train_ratio * len(balanced_items))
```

```
val_size = len(balanced_items) - train_size
```

```
train_items = balanced_items[:train_size]
```

```
val_items = balanced_items[train_size:]
```

```
train_data = [get_item(item, is_val=False) for item in train_items]
```

```
val_data = [get_item(item, is_val=True) for item in val_items]
```

```
train_loader = DataLoader(train_data, batch_size=batch_size, shuffle=True)
```

```
val_loader = DataLoader(val_data, batch_size=batch_size, shuffle=False)
```

```
print(f'\nTrain (OC): {len(train_data)} иконок")
```

```
print(f'Val (OC): {len(val_data)} иконок")
```

```
train_labels = [label for _, label in train_data]
```

```
label_counts = {}
```

```
for s in train_labels:
```

```
    name = idx_to_os[s]
```

```

label_counts[name] = label_counts.get(name, 0) + 1

print(f"\nРаспределение по ОС в Train (после балансировки):")

for name, count in sorted(label_counts.items()):

    print(f" {name}: {count}")

return train_loader, val_loader, num_os, idx_to_os

def findDirection():

    base_path = os.path.dirname(__file__)

    DATASET_PATH = os.path.join(base_path, "НСвОИиТ курсовая
датасет_converted")

    LABELS_FILE = os.path.join(DATASET_PATH, "labels_remapped.json")

    MODEL_PATH = os.path.join(base_path, "direction_model_best.pth")

    BATCH_SIZE = 128

    EPOCHS = 30

    LEARNING_RATE = 0.001

    IMAGE_SIZE = 128

    DEVICE = torch.device("cuda" if torch.cuda.is_available() else "cpu")

    print(f"\nУстройство: {DEVICE}")

```

```

if os.path.exists(MODEL_PATH):

    print(f"\nНайдена сохранённая модель: {MODEL_PATH}")

    answer = input("Загрузить сохранённую модель? (y/n): ")

    if answer.lower() == 'y':

        train_loader, val_loader, num_directions, idx_to_direction = loadDirection(

            labels_file=LABELS_FILE,

            root_dir=DATASET_PATH,

            batch_size=BATCH_SIZE,

            train_ratio=0.8,

            image_size=IMAGE_SIZE

        )

        direction_names = list(idx_to_direction.values())

        model = models.resnet18(weights=None)

        model.conv1 = nn.Conv2d(3, 64, kernel_size=7, stride=2, padding=3,
bias=False)

        in_features = model.fc.in_features

        model.fc = nn.Linear(in_features, num_directions)

        model.load_state_dict(torch.load(MODEL_PATH,
map_location=DEVICE))

        model = model.to(DEVICE)

```

```

print(f"Модель загружена: {MODEL_PATH}")

def predict(image):

    model.eval()

    transform = transforms.Compose([

        transforms.Resize((IMAGE_SIZE, IMAGE_SIZE)),

        transforms.ToTensor(),

        transforms.Normalize(mean=[0.485, 0.456, 0.406],

                               std=[0.229, 0.224, 0.225])

    ])

    image_tensor = transform(image).unsqueeze(0).to(DEVICE)

    with torch.no_grad():

        output = model(image_tensor)

        probabilities = torch.softmax(output, dim=1)

        probs = {direction_names[i]: probabilities[0][i].item() for i in
range(num_directions)}

    return probs

return predict

train_loader, val_loader, num_directions, idx_to_direction = loadDirection(

    labels_file=LABELS_FILE,

    root_dir=DATASET_PATH,

    batch_size=BATCH_SIZE,

```



```

    train_ratio=0.8,

    image_size=IMAGE_SIZE
)

all_val_directions = []

for _, direction_labels in val_loader:

    all_val_directions.extend(direction_labels.numpy())

unique_directions = sorted(set(all_val_directions))

direction_names = [idx_to_direction[i] for i in unique_directions]

plt.figure(figsize=(8, 6))

plt.hist(all_val_directions,          bins=len(unique_directions),          alpha=0.7,
color='purple', edgecolor='black')

plt.xticks(unique_directions, direction_names, rotation=45)

plt.title("Распределение направлений дизайна в валидационной выборке")

plt.xlabel("Направление")

plt.ylabel("Количество иконок")

plt.tight_layout()

plt.savefig("direction_distribution.png")

plt.close()

print("Гистограмма направлений сохранена: direction_distribution.png")

```

```

model = models.resnet18(weights=models.ResNet18_Weights.IMAGENET1K_V1)

model.conv1 = nn.Conv2d(3, 64, kernel_size=7, stride=2, padding=3, bias=False)

in_features = model.fc.in_features

model.fc = nn.Linear(in_features, num_directions)

model = model.to(DEVICE)

criterion = nn.CrossEntropyLoss()

optimizer = optim.Adam(model.parameters(), lr=LEARNING_RATE)

scheduler = optim.lr_scheduler.ReduceLROnPlateau(optimizer, mode='max',
patience=5, factor=0.5)

print("\nОбучение модели для определения направления дизайна...")

train_losses = []

val_accuracies = []

best_acc = 0

best_epoch = 0

best_model_state = None

patience = 7

patience_counter = 0

all_predictions = []

all_labels = []

```

```

for epoch in range(EPOCHS):

    model.train()

    running_loss = 0.0

    for images, direction_labels in train_loader:

        images = images.to(DEVICE)

        direction_labels = direction_labels.to(DEVICE)

        optimizer.zero_grad()

        outputs = model(images)

        loss = criterion(outputs, direction_labels)

        loss.backward()

        optimizer.step()

    running_loss += loss.item()

    avg_loss = running_loss / len(train_loader)

    train_losses.append(avg_loss)

    model.eval()

    correct = 0

```

```

total = 0

epoch_predictions = []

epoch_labels = []

with torch.no_grad():

    for images, direction_labels in val_loader:

        images = images.to(DEVICE)

        direction_labels = direction_labels.to(DEVICE)

        outputs = model(images)

        _, predicted = torch.max(outputs, 1)

        total += direction_labels.size(0)

        correct += (predicted == direction_labels).sum().item()


    epoch_predictions.extend(predicted.cpu().numpy())

    epoch_labels.extend(direction_labels.cpu().numpy())


acc = 100 * correct / total

val_accuracies.append(acc)

scheduler.step(acc)

if acc > best_acc:

```

```

best_acc = acc

best_epoch = epoch + 1

best_model_state = model.state_dict().copy()

all_predictions = epoch_predictions

all_labels = epoch_labels

patience_counter = 0

print(f'Epoch {epoch+1}/{EPOCHS} | Loss: {avg_loss:.4f} | Val Acc:
{acc:.2f}% – Новый лучший')

else:

    patience_counter += 1

    print(f'Epoch {epoch+1}/{EPOCHS} | Loss: {avg_loss:.4f} | Val Acc:
{acc:.2f}% (лучший: {best_acc:.2f}%)')

if patience_counter >= patience:

    print(f'\nEarly stopping на эпохе {epoch+1}')

    break

if best_model_state is not None:

    model.load_state_dict(best_model_state)

    print(f'\nЗагружена лучшая модель с эпохи {best_epoch}, точность
{best_acc:.2f}%)

print(f'\nФинальная точность: {best_acc:.2f}%)

```

```

plt.figure(figsize=(8, 5))

plt.plot(range(1, len(train_losses)+1), train_losses, marker='o', linewidth=2,
markersize=4)

plt.title("График потерь при обучении (определение направления дизайна)")

plt.xlabel("Эпоха")

plt.ylabel("Loss")

plt.grid(True)

plt.savefig("direction_loss.png")

plt.close()

print("График потерь сохранён: direction_loss.png")

```

```

plt.figure(figsize=(8, 5))

plt.plot(range(1, len(val_accuracies)+1), val_accuracies, marker='o', linewidth=2,
markersize=4, color='green')

plt.axhline(y=best_acc, color='r', linestyle='--', label=f'Best: {best_acc:.2f}%')

plt.title("График точности на валидации (определение направления
дизайна)")

plt.xlabel("Эпоха")

plt.ylabel("Точность (%)")

plt.grid(True)

plt.legend()

plt.savefig("direction_accuracy.png")

```

```

plt.close()

print("График точности сохранён: direction_accuracy.png")

class_names = direction_names

cm = confusion_matrix(all_labels, all_predictions)

plt.figure(figsize=(8, 6))

plt.imshow(cm, interpolation='nearest', cmap='Blues')

plt.title(f'Confusion Matrix (Направления дизайна) - Лучшая точность:
{best_acc:.2f}%')

plt.colorbar()

tick_marks = np.arange(len(class_names))

plt.xticks(tick_marks, class_names, rotation=45, ha='right')

plt.yticks(tick_marks, class_names)

for i in range(cm.shape[0]):
    for j in range(cm.shape[1]):
        plt.text(j, i, str(cm[i, j]),
                  horizontalalignment="center",
                  color="white" if cm[i, j] > cm.max() / 2 else "black")

plt.xlabel("Предсказано")

```

```

plt.ylabel("Истина")

plt.tight_layout()

plt.savefig("direction_confusion_matrix.png")

plt.close()

print("Confusion matrix сохранена: direction_confusion_matrix.png")


report = classification_report(all_labels, all_predictions,
target_names=class_names)

print("\nОтчёт классификации (Направления дизайна):")

print(report)


with open("direction_report.txt", "w", encoding="utf-8") as f:

    f.write(report)

print("Отчёт сохранён: direction_report.txt")


torch.save(model.state_dict(), MODEL_PATH)

print(f"Модель сохранена: {MODEL_PATH}")


def predict(image):

    model.eval()

    transform = transforms.Compose([

        transforms.Resize((IMAGE_SIZE, IMAGE_SIZE)),

        transforms.ToTensor(),

```



```

        transforms.Normalize(mean=[0.485, 0.456, 0.406],
                               std=[0.229, 0.224, 0.225])
    ])

    image_tensor = transform(image).unsqueeze(0).to(DEVICE)

    with torch.no_grad():

        output = model(image_tensor)

        probabilities = torch.softmax(output, dim=1)

        probs = {direction_names[i]: probabilities[0][i].item() for i in
range(num_directions)}

    return probs

return predict

def findStyle():

    base_path = os.path.dirname(__file__)

    DATASET_PATH = os.path.join(base_path, "НСВОИИТ курсовая
датасет_converted")

    LABELS_FILE = os.path.join(DATASET_PATH, "labels_remapped.json")

    MODEL_PATH = os.path.join(base_path, "style_model_best.pth")

    BATCH_SIZE = 128

    EPOCHS = 30

    LEARNING_RATE = 0.001

```

```

IMAGE_SIZE = 128

DEVICE = torch.device("cuda" if torch.cuda.is_available() else "cpu")

# Словарь: направление → список стилей

DIRECTION_TO_STYLES = {

    "Skeuomorphism": ["Apple Skeuomorphism", "Windows Aero"],

    "Flat": ["Apple Flat by Jony Ive", "Windows Fluent", "Windows Metro"],

    "Glassmorphism": ["Apple Liquid Glass"],

    "Retro": ["Windows 80-90s Retro"]

}

# Обратный словарь: стиль → направление

STYLE_TO_DIRECTION = {}

for direction, styles in DIRECTION_TO_STYLES.items():

    for style in styles:

        STYLE_TO_DIRECTION[style] = direction

print(f"\nУстройство: {DEVICE}")

if os.path.exists(MODEL_PATH):

    print(f"\nНайдена сохранённая модель: {MODEL_PATH}")

    answer = input("Загрузить сохранённую модель? (y/n): ")

```

```

if answer.lower() == 'y':

    train_loader, val_loader, num_styles, idx_to_style = loadStyle(

        labels_file=LABELS_FILE,

        root_dir=DATASET_PATH,

        batch_size=BATCH_SIZE,

        train_ratio=0.8,

        image_size=IMAGE_SIZE

    )


    model = models.resnet18(weights=None)

    model.conv1 = nn.Conv2d(3, 64, kernel_size=7, stride=2, padding=3,
bias=False)

    in_features = model.fc.in_features

    model.fc = nn.Linear(in_features, num_styles)

    model.load_state_dict(torch.load(MODEL_PATH,
map_location=DEVICE))

    model = model.to(DEVICE)


    print(f"Модель загружена: {MODEL_PATH}")


    style_names = list(idx_to_style.values())


    def predict(image, direction=None):

```

```

model.eval()

transform = transforms.Compose([

    transforms.Resize((IMAGE_SIZE, IMAGE_SIZE)),

    transforms.ToTensor(),

    transforms.Normalize(mean=[0.485, 0.456, 0.406],
                           std=[0.229, 0.224, 0.225])

])

image_tensor = transform(image).unsqueeze(0).to(DEVICE)

with torch.no_grad():

    output = model(image_tensor)

    probabilities = torch.softmax(output, dim=1)

    probs = {style_names[i]: probabilities[0][i].item() for i in
range(num_styles)}


# Если передано направление, фильтруем стили
if direction is not None:

    allowed_styles = DIRECTION_TO_STYLES.get(direction, [])

    if allowed_styles:

        probs = {s: p for s, p in probs.items() if s in allowed_styles}

        # Нормализуем вероятности

        total = sum(probs.values())

        if total > 0:

            probs = {s: p / total for s, p in probs.items()}

```

```

        return probs

    return predict

train_loader, val_loader, num_styles, idx_to_style = loadStyle(

    labels_file=LABELS_FILE,

    root_dir=DATASET_PATH,

    batch_size=BATCH_SIZE,

    train_ratio=0.8,

    image_size=IMAGE_SIZE

)

all_val_styles = []

for _, style_labels in val_loader:

    all_val_styles.extend(style_labels.numpy())

unique_styles = sorted(set(all_val_styles))

style_names_list = [idx_to_style[i] for i in unique_styles]

plt.figure(figsize=(10, 6))

plt.hist(all_val_styles, bins=len(unique_styles), alpha=0.7, color='green',
edgecolor='black')

plt.xticks(unique_styles, style_names_list, rotation=45, fontsize=10)

```

```

plt.title("Распределение стилей в валидационной выборке")

plt.xlabel("Стиль")

plt.ylabel("Количество иконок")

plt.tight_layout()

plt.savefig("style_distribution.png")

plt.close()

print("Гистограмма стилей сохранена: style_distribution.png")


model =
models.resnet18(weights=models.ResNet18_Weights.IMAGENET1K_V1)

model.conv1 = nn.Conv2d(3, 64, kernel_size=7, stride=2, padding=3, bias=False)

in_features = model.fc.in_features

model.fc = nn.Linear(in_features, num_styles)

model = model.to(DEVICE)


criterion = nn.CrossEntropyLoss()

optimizer = optim.Adam(model.parameters(), lr=LEARNING_RATE)

scheduler = optim.lr_scheduler.ReduceLROnPlateau(optimizer, mode='max',
patience=3, factor=0.5)


print("\nОбучение модели для определения стиля...")

print(f"Количество классов стилей: {num_styles}")

```

```
train_losses = []

val_accuracies = []

best_acc = 0

best_epoch = 0

best_model_state = None

patience = 5

patience_counter = 0

all_predictions = []

all_labels = []


for epoch in range(EPOCHS):

    model.train()

    running_loss = 0.0


    for images, style_labels in train_loader:

        images = images.to(DEVICE)

        style_labels = style_labels.to(DEVICE)


        optimizer.zero_grad()

        outputs = model(images)

        loss = criterion(outputs, style_labels)

        loss.backward()
```

```
optimizer.step()
```

```
running_loss += loss.item()
```

```
avg_loss = running_loss / len(train_loader)
```

```
train_losses.append(avg_loss)
```

```
model.eval()
```

```
correct = 0
```

```
total = 0
```

```
epoch_predictions = []
```

```
epoch_labels = []
```

```
with torch.no_grad():
```

```
    for images, style_labels in val_loader:
```

```
        images = images.to(DEVICE)
```

```
        style_labels = style_labels.to(DEVICE)
```

```
        outputs = model(images)
```

```
        _, predicted = torch.max(outputs, 1)
```

```
        total += style_labels.size(0)
```

```
        correct += (predicted == style_labels).sum().item()
```



```

epoch_predictions.extend(predicted.cpu().numpy())

epoch_labels.extend(style_labels.cpu().numpy())


acc = 100 * correct / total

val_accuracies.append(acc)


scheduler.step(acc)


if acc > best_acc:

    best_acc = acc

    best_epoch = epoch + 1

    best_model_state = model.state_dict().copy()

    all_predictions = epoch_predictions

    all_labels = epoch_labels

    patience_counter = 0

    print(f'Epoch {epoch+1}/{EPOCHS} | Loss: {avg_loss:.4f} | Val Acc:
{acc:.2f}% – НОВЫЙ ЛУЧШИЙ')

    else:

        patience_counter += 1

        print(f'Epoch {epoch+1}/{EPOCHS} | Loss: {avg_loss:.4f} | Val Acc:
{acc:.2f}% (лучший: {best_acc:.2f}%)')


if patience_counter >= patience:

```

```

print(f"\nРанняя остановка на эпохе {epoch+1}")

break

if best_model_state is not None:

    model.load_state_dict(best_model_state)

    print(f"\nЗагружена лучшая модель с эпохи {best_epoch}, точность
{best_acc:.2f}%")

print(f"\nФинальная точность: {best_acc:.2f}%")

plt.figure(figsize=(8, 5))

plt.plot(range(1, len(train_losses)+1), train_losses, marker='o', linewidth=2,
markersize=4)

plt.title("График потерь при обучении (определение стиля)")

plt.xlabel("Эпоха")

plt.ylabel("Loss")

plt.grid(True)

plt.savefig("style_loss.png")

plt.close()

print("График потерь сохранён: style_loss.png")

plt.figure(figsize=(8, 5))

```

```
plt.plot(range(1, len(val_accuracies)+1), val_accuracies, marker='o', linewidth=2,  
markersize=4, color='green')
```

```
plt.axhline(y=best_acc, color='r', linestyle='--', label=f'Best: {best_acc:.2f}%')
```

```
plt.title("График точности на валидации (определение стиля)")
```

```
plt.xlabel("Эпоха")
```

```
plt.ylabel("Точность (%)")
```

```
plt.grid(True)
```

```
plt.legend()
```

```
plt.savefig("style_accuracy.png")
```

```
plt.close()
```

```
print("График точности сохранён: style_accuracy.png")
```

```
class_names = style_names_list
```

```
cm = confusion_matrix(all_labels, all_predictions)
```

```
plt.figure(figsize=(10, 8))
```

```
plt.imshow(cm, interpolation='nearest', cmap='Blues')
```

```
plt.title(f'Confusion Matrix (Стили) - Лучшая точность: {best_acc:.2f}%')
```

```
plt.colorbar()
```

```
tick_marks = np.arange(len(class_names))
```

```
plt.xticks(tick_marks, class_names, rotation=45, ha='right')
```

```
plt.yticks(tick_marks, class_names)
```

```

for i in range(cm.shape[0]):
    for j in range(cm.shape[1]):
        plt.text(j, i, str(cm[i, j]),
                  horizontalalignment="center",
                  color="white" if cm[i, j] > cm.max() / 2 else "black")

plt.xlabel("Предсказано")
plt.ylabel("Истина")
plt.tight_layout()
plt.savefig("style_confusion_matrix.png")
plt.close()

print("Confusion matrix сохранена: style_confusion_matrix.png")

with open("style_report.txt", "w", encoding="utf-8") as f:
    f.write(classification_report(all_labels,
                                all_predictions,
                                target_names=class_names))

print("Отчёт сохранён: style_report.txt")

torch.save(model.state_dict(), MODEL_PATH)

print(f"Модель сохранена: {MODEL_PATH}")

style_names = list(idx_to_style.values())

```

```

def predict(image, direction=None):

    model.eval()

    transform = transforms.Compose([

        transforms.Resize((IMAGE_SIZE, IMAGE_SIZE)),

        transforms.ToTensor(),

        transforms.Normalize(mean=[0.485, 0.456, 0.406],

                               std=[0.229, 0.224, 0.225])

    ])

    image_tensor = transform(image).unsqueeze(0).to(DEVICE)

    with torch.no_grad():

        output = model(image_tensor)

        probabilities = torch.softmax(output, dim=1)

        probs = {style_names[i]: probabilities[0][i].item() for i in
range(num_styles)}

    # Если передано направление, фильтруем стили

    if direction is not None:

        allowed_styles = DIRECTION_TO_STYLES.get(direction, [])

        if allowed_styles:

            probs = {s: p for s, p in probs.items() if s in allowed_styles}

            total = sum(probs.values())

            if total > 0:

```

```

        probs = {s: p / total for s, p in probs.items()}

    return probs

return predict

def findOS():

    base_path = os.path.dirname(__file__)

    DATASET_PATH = os.path.join(base_path, "НСВОИИТ курсовая
датасет_converted")

    LABELS_FILE = os.path.join(DATASET_PATH, "labels_remapped.json")

    MODEL_PATH = os.path.join(base_path, "os_model_best.pth")

    BATCH_SIZE = 128

    EPOCHS = 30

    LEARNING_RATE = 0.001

    IMAGE_SIZE = 128

    DEVICE = torch.device("cuda" if torch.cuda.is_available() else "cpu")

    # Словарь: стиль → список ОС

    STYLE_TO_OS = {

        "Apple Skeuomorphism": [

            "iPhone OS",

```

```
"iOS 6",  
  
"Mac OS X 10.0 Cheetah",  
  
"Mac OS X 10.5 Leopard",  
  
"Mac OS X 10.9 Mavericks"  
  
],  
  
"Windows Aero": [  
  
    "Windows Vista",  
  
    "Windows 7"  
  
],  
  
"Apple Flat by Jony Ive": [  
  
    "iOS 7",  
  
    "iOS 11",  
  
    "iOS 18",  
  
    "Mac OS X 10.10 Yosemite",  
  
    "macOS 10.15 Catalina",  
  
    "macOS 11 Big Sur"  
  
],  
  
"Windows Fluent": [  
  
    "Windows 11"  
  
],  
  
"Windows Metro": [  
  
    "Windows 10"
```

```

    ],
    "Apple Liquid Glass": [
        "iOS 26",
        "macOS 26 Tahoe"
    ],
    "Windows 80-90s Retro": [
        "Windows 3.0",
        "Windows 98"
    ]
}

# Обратный словарь: ОС → стиль (для проверки)
OS_TO_STYLE = {}

for style, os_list in STYLE_TO_OS.items():
    for os_name in os_list:
        OS_TO_STYLE[os_name] = style

print(f"\nУстройство: {DEVICE}")

if os.path.exists(MODEL_PATH):
    print(f"\nНайдена сохранённая модель: {MODEL_PATH}")
    answer = input("Загрузить сохранённую модель? (y/n): ")

```



```

if answer.lower() == 'y':

    train_loader, val_loader, num_os, idx_to_os = loadOS(

        labels_file=LABELS_FILE,

        root_dir=DATASET_PATH,

        batch_size=BATCH_SIZE,

        train_ratio=0.8,

        image_size=IMAGE_SIZE

    )


    model = models.resnet18(weights=None)

    model.conv1 = nn.Conv2d(3, 64, kernel_size=7, stride=2, padding=3,
bias=False)

    in_features = model.fc.in_features

    model.fc = nn.Linear(in_features, num_os)

    model.load_state_dict(torch.load(MODEL_PATH,
map_location=DEVICE))

    model = model.to(DEVICE)


    print(f"Модель загружена: {MODEL_PATH}")


    os_names = list(idx_to_os.values())


    def predict(image, style=None):

```

```

model.eval()

transform = transforms.Compose([

    transforms.Resize((IMAGE_SIZE, IMAGE_SIZE)),

    transforms.ToTensor(),

    transforms.Normalize(mean=[0.485, 0.456, 0.406],
                           std=[0.229, 0.224, 0.225])

])

image_tensor = transform(image).unsqueeze(0).to(DEVICE)

with torch.no_grad():

    output = model(image_tensor)

    probabilities = torch.softmax(output, dim=1)

    probs = {os_names[i]: probabilities[0][i].item() for i in range(num_os)}

# Если передан стиль, фильтруем ОС

if style is not None:

    allowed_os = STYLE_TO_OS.get(style, [])

    if allowed_os:

        probs = {os: p for os, p in probs.items() if os in allowed_os}

        total = sum(probs.values())

        if total > 0:

            probs = {os: p / total for os, p in probs.items()}

```

```

        return probs

    return predict

train_loader, val_loader, num_os, idx_to_os = loadOS(

    labels_file=LABELS_FILE,

    root_dir=DATASET_PATH,

    batch_size=BATCH_SIZE,

    train_ratio=0.8,

    image_size=IMAGE_SIZE

)

all_val_os = []

for _, os_labels in val_loader:

    all_val_os.extend(os_labels.numpy())

unique_os = sorted(set(all_val_os))

os_names_list = [idx_to_os[i] for i in unique_os]

plt.figure(figsize=(14, 6))

plt.hist(all_val_os,      bins=len(unique_os),      alpha=0.7,      color='blue',
edgecolor='black')

plt.xticks(unique_os, os_names_list, rotation=90, fontsize=8)

plt.title("Распределение ОС в валидационной выборке")

```

```

plt.xlabel("ОС")

plt.ylabel("Количество иконок")

plt.tight_layout()

plt.savefig("os_distribution.png")

plt.close()

print("Гистограмма ОС сохранена: os_distribution.png")


model = models.resnet18(weights=models.ResNet18_Weights.IMAGENET1K_V1)

model.conv1 = nn.Conv2d(3, 64, kernel_size=7, stride=2, padding=3, bias=False)

in_features = model.fc.in_features

model.fc = nn.Linear(in_features, num_os)

model = model.to(DEVICE)


criterion = nn.CrossEntropyLoss()

optimizer = optim.Adam(model.parameters(), lr=LEARNING_RATE)

scheduler = optim.lr_scheduler.ReduceLROnPlateau(optimizer, mode='max',
patience=3, factor=0.5)


print("\nОбучение модели для определения ОС...")

print(f"Количество классов ОС: {num_os}")


train_losses = []

```

```
val_accuracies = []

best_acc = 0

best_epoch = 0

best_model_state = None

patience = 5

patience_counter = 0

all_predictions = []

all_labels = []


for epoch in range(EPOCHS):

    model.train()

    running_loss = 0.0


    for images, os_labels in train_loader:

        images = images.to(DEVICE)

        os_labels = os_labels.to(DEVICE)


        optimizer.zero_grad()

        outputs = model(images)

        loss = criterion(outputs, os_labels)

        loss.backward()

        optimizer.step()
```

```

    running_loss += loss.item()

avg_loss = running_loss / len(train_loader)
train_losses.append(avg_loss)

model.eval()

correct = 0

total = 0

epoch_predictions = []
epoch_labels = []

with torch.no_grad():
    for images, os_labels in val_loader:

        images = images.to(DEVICE)

        os_labels = os_labels.to(DEVICE)

        outputs = model(images)

        _, predicted = torch.max(outputs, 1)

        total += os_labels.size(0)

        correct += (predicted == os_labels).sum().item()

    epoch_predictions.extend(predicted.cpu().numpy())

```

```

epoch_labels.extend(os_labels.cpu().numpy())

acc = 100 * correct / total

val_accuracies.append(acc)

scheduler.step(acc)

if acc > best_acc:

    best_acc = acc

    best_epoch = epoch + 1

    best_model_state = model.state_dict().copy()

    all_predictions = epoch_predictions

    all_labels = epoch_labels

    patience_counter = 0

    print(f"Epoch {epoch+1}/{EPOCHS} | Loss: {avg_loss:.4f} | Val Acc:
{acc:.2f}% – НОВЫЙ ЛУЧШИЙ")

    else:

        patience_counter += 1

        print(f"Epoch {epoch+1}/{EPOCHS} | Loss: {avg_loss:.4f} | Val Acc:
{acc:.2f}% (лучший: {best_acc:.2f}%)")

if patience_counter >= patience:

    print(f"\nРанняя остановка на эпохе {epoch+1}")

```

```

break

if best_model_state is not None:

    model.load_state_dict(best_model_state)

    print(f"\nЗагружена лучшая модель с эпохи {best_epoch}, точность
{best_acc:.2f}%")

print(f"\nФинальная точность: {best_acc:.2f}%")

plt.figure(figsize=(8, 5))

plt.plot(range(1, len(train_losses)+1), train_losses, marker='o', linewidth=2,
markersize=4)

plt.title("График потерь при обучении (определение ОС)")

plt.xlabel("Эпоха")

plt.ylabel("Loss")

plt.grid(True)

plt.savefig("os_loss.png")

plt.close()

print("График потерь сохранён: os_loss.png")

plt.figure(figsize=(8, 5))

plt.plot(range(1, len(val_accuracies)+1), val_accuracies, marker='o', linewidth=2,
markersize=4, color='green')

```



```

plt.axhline(y=best_acc, color='r', linestyle='--', label=f'Best: {best_acc:.2f}%')

plt.title("График точности на валидации (определение ОС)")

plt.xlabel("Эпоха")

plt.ylabel("Точность (%)")

plt.grid(True)

plt.legend()

plt.savefig("os_accuracy.png")

plt.close()

print("График точности сохранён: os_accuracy.png")


class_names = [idx_to_os[i] for i in range(num_os)]

cm = confusion_matrix(all_labels, all_predictions)


plt.figure(figsize=(min(20, num_os * 0.6), min(16, num_os * 0.5)))

plt.imshow(cm, interpolation='nearest', cmap='Blues')

plt.title(f'Confusion Matrix (ОС) - Лучшая точность: {best_acc:.2f}%')

plt.colorbar()


tick_marks = np.arange(len(class_names))

plt.xticks(tick_marks, class_names, rotation=90, ha='right', fontsize=7)

plt.yticks(tick_marks, class_names, fontsize=7)

```

```

for i in range(cm.shape[0]):
    for j in range(cm.shape[1]):
        if cm[i, j] > 0:
            plt.text(j, i, str(cm[i, j]),
                     horizontalalignment="center",
                     fontsize=6,
                     color="white" if cm[i, j] > cm.max() / 2 else "black")

plt.xlabel("Предсказано")
plt.ylabel("Истина")
plt.tight_layout()
plt.savefig("os_confusion_matrix.png")
plt.close()

print("Confusion matrix сохранена: os_confusion_matrix.png")

# Частые ошибки

from collections import Counter

print("10 самых частых ошибок")

errors = []

for true, pred in zip(all_labels, all_predictions):
    if true != pred:

```

```

errors.append((true, pred))

error_pairs = Counter(errors)

for (true, pred), count in error_pairs.most_common(10):

    print(f" {class_names[true]} → {class_names[pred]}: {count} раз(a)")

with open("os_report.txt", "w", encoding="utf-8") as f:

    f.write(classification_report(all_labels,                                all_predictions,
target_names=class_names))

print("\nПолный отчёт сохранён: os_report.txt")

torch.save(model.state_dict(), MODEL_PATH)

print(f"Модель сохранена: {MODEL_PATH}")

os_names = list(idx_to_os.values())

def predict(image, style=None):

    model.eval()

    transform = transforms.Compose([

        transforms.Resize((IMAGE_SIZE, IMAGE_SIZE)),

        transforms.ToTensor(),

        transforms.Normalize(mean=[0.485, 0.456, 0.406],

                                std=[0.229, 0.224, 0.225])

```

```

])

image_tensor = transform(image).unsqueeze(0).to(DEVICE)

with torch.no_grad():

    output = model(image_tensor)

    probabilities = torch.softmax(output, dim=1)

    probs = {os_names[i]: probabilities[0][i].item() for i in range(num_os)}

# Если передан стиль, фильтруем ОС

if style is not None:

    allowed_os = STYLE_TO_OS.get(style, [])

    if allowed_os:

        probs = {os: p for os, p in probs.items() if os in allowed_os}

        total = sum(probs.values())

        if total > 0:

            probs = {os: p / total for os, p in probs.items()}

    return probs

return predict

def AI():

    base_path = os.path.dirname(__file__)

```

```
DATASET_PATH = os.path.join(base_path, "НСВОИИТ курсовая  
датасет_converted")
```

```
LABELS_FILE = os.path.join(DATASET_PATH, "labels_remapped.json")
```

```
predict_direction = findDirection()
```

```
predict_style = findStyle()
```

```
predict_os = findOS()
```

```
images, paths = user()
```

```
if images:
```

```
    print(f"\nКлассификация пользовательских иконок:\n")
```

```
    for img, path in zip(images, paths):
```

```
        filename = os.path.basename(path)
```

```
        # Шаг 1: Определяем направление
```

```
        direction_probs = predict_direction(img)
```

```
        best_direction = max(direction_probs, key=direction_probs.get)
```

```
        # Шаг 2: Определяем стиль с учётом направления
```

```
        style_probs = predict_style(img, direction=best_direction)
```

```
        best_style = max(style_probs, key=style_probs.get) if style_probs else None
```

```

# Шаг 3: Определяем ОС с учётом стиля

os_probs = predict_os(img, style=best_style)

best_os = max(os_probs, key=os_probs.get) if os_probs else None


print(f"\n{filename}")

print("-" * 50)

print(f"        Направление  дизайна:  {best_direction}    (уверенность
{direction_probs[best_direction]*100:.1f}%)")


if best_style:

    print(f"                СТИЛЬ:                {best_style}            (уверенность
{style_probs[best_style]*100:.1f}%)")


if best_os:

    print(f"  ОС: {best_os} (уверенность {os_probs[best_os]*100:.1f}%)")


# Детализация

sorted_directions = sorted(direction_probs.items(), key=lambda x: x[1],
reverse=True)

print(f"\n Детализация по направлениям:")

for i, (direction, prob) in enumerate(sorted_directions):

    print(f"    {i+1}. {direction}: {prob*100:.1f}%)

```

```

if style_probs:

    sorted_styles = sorted(style_probs.items(), key=lambda x: x[1],
reverse=True)

    print(f"\n Детализация по стилям (топ-5):")

    for i, (style, prob) in enumerate(sorted_styles[:5]):

        print(f"    {i+1}. {style}: {prob*100:.1f}%")


if os_probs:

    sorted_os = sorted(os_probs.items(), key=lambda x: x[1], reverse=True)

    print(f"\n Детализация по ОС (топ-5):")

    for i, (os_name, prob) in enumerate(sorted_os[:5]):

        print(f"    {i+1}. {os_name}: {prob*100:.1f}%")


else:

    print("\nНет изображений для классификации")

    print("Положите иконки в папку User на рабочем столе и запустите снова")

```

AI()